

NeRF: Representing Scenes as Neural Radiance Fields for View Synthesis



Ben Mildenhall*



UC Berkeley



Pratul Srinivasan*



UC Berkeley



Matt Tancik*



UC Berkeley



Jon Barron



Google Research



Ravi Ramamoorthi



UC San Diego



Ren Ng



UC Berkeley



Lots of recent progress using deep learning for view synthesis!



Wiles CVPR 2020



Choi ICCV 2019

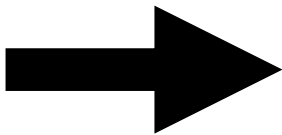


Flynn CVPR 2019

The problem of novel view interpolation

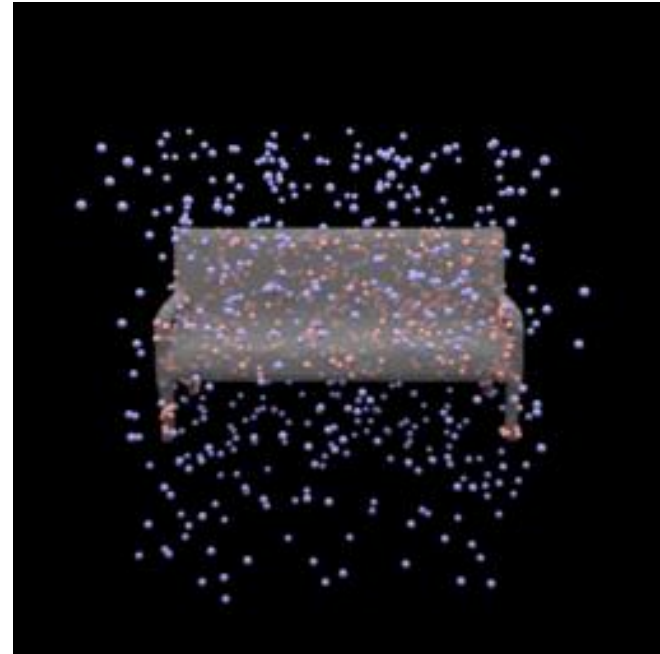
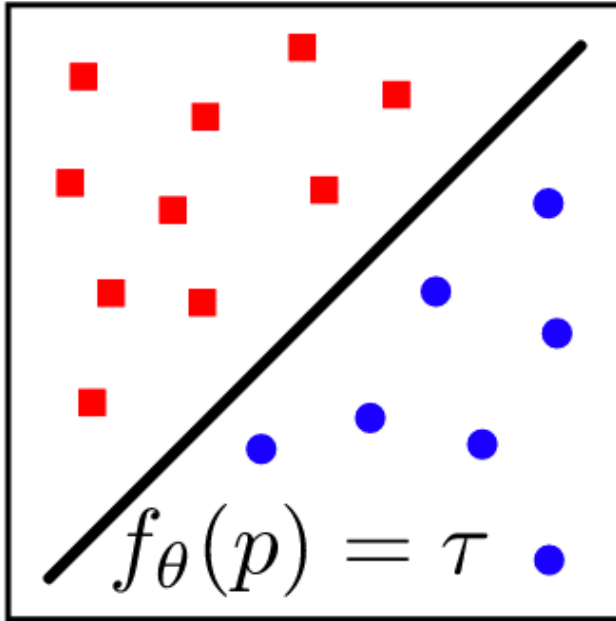


Inputs: sparsely sampled images of scene



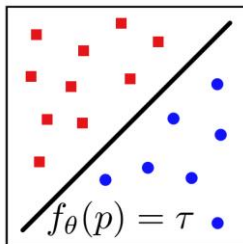
Outputs: new views of same scene

Neural networks as a continuous shape representation

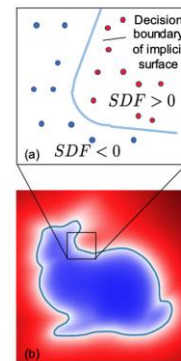


Neural networks as a continuous shape representation

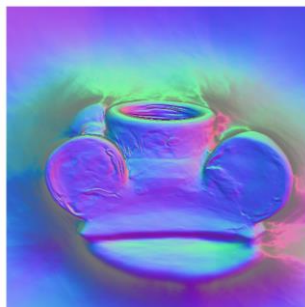
Occupancy Networks
(Mescheder et al. 2019)
 $(x, y, z) \rightarrow \textit{occupancy}$



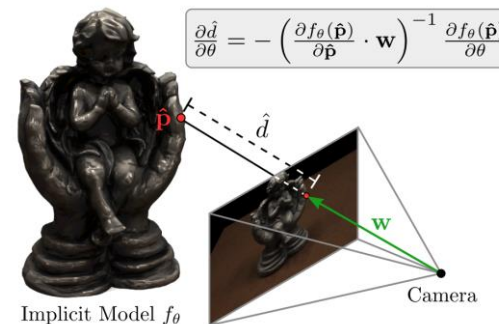
DeepSDF
(Park et al. 2019)
 $(x, y, z) \rightarrow \textit{distance}$



Scene Representation Networks
(Sitzmann et al. 2019)
 $(x, y, z) \rightarrow \textit{latent vec. (color, dist.)}$



Differentiable Volumetric Rendering
(Niemeyer et al. 2020)
 $(x, y, z) \rightarrow \textit{color, occ.}$



Neural networks as a continuous shape representation

DeepSDF

(Park et al. 2019)

$(x, y, z) \rightarrow \text{distance}$



Occupancy Networks

(Mescheder et al. 2019)

$(x, y, z) \rightarrow \text{occupancy}$



- Limited rendering model: difficult to optimize
- + Highly compressible (1-10 MB)

Scene Representation Networks

(Sitzmann et al. 2019)

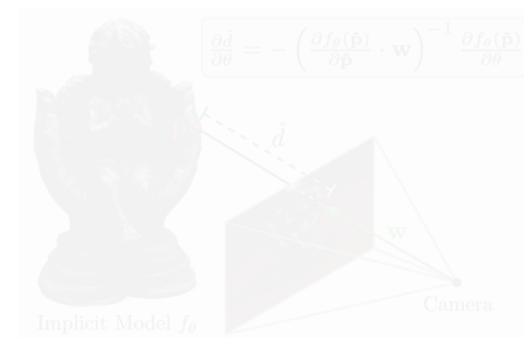
$(x, y, z) \rightarrow \text{latent vec. (color, dist.)}$



Differentiable Volumetric Rendering

(Niemeyer et al. 2020)

$(x, y, z) \rightarrow \text{color, occ.}$



NeRF (neural radiance fields):

Neural networks as a volume representation, using volume rendering to do view synthesis.

$$(x, y, z, \theta, \phi) \rightarrow \textit{color, opacity}$$

NO

LLFF [Mildenhall 2019]



NeRF



S

Nearest Input



NeRF achieves state-of-the-art results on an extremely difficult problem

Neural Volumes [Lombardi 2019]



NeRF



Nearest Input

NeRF achieves state-of-the-art results on an extremely difficult problem

SRN [Sitzmann 2019]



NeRF



Nearest Input

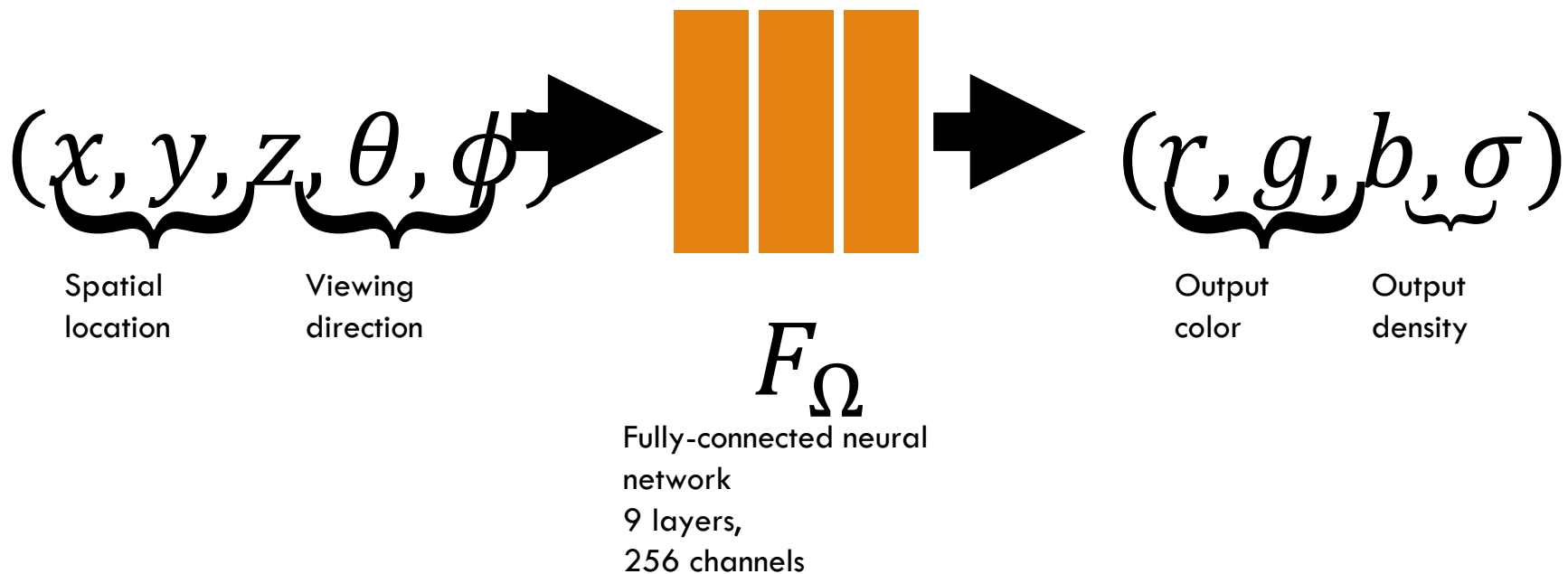
Key points

- ▶ Continuous neural network as a volumetric scene representation (5D = xyz + direction)
- ▶ Use volume rendering model to synthesize new views
- ▶ Optimize using rendering loss for one scene (no prior training)
- ▶ One extra trick for passing coordinates into network to get high frequency details

Key points

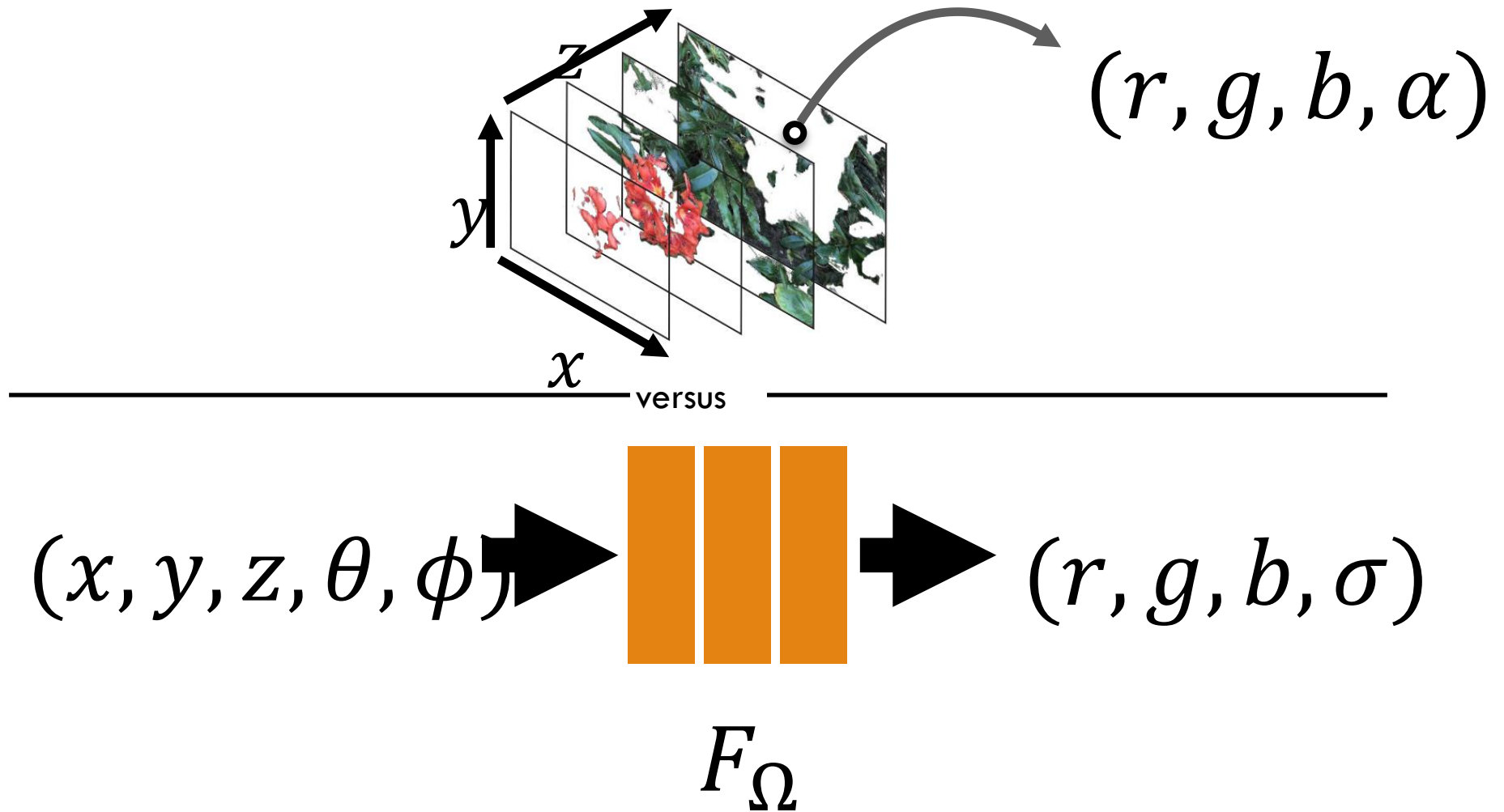
- ▶ Continuous neural network as a volumetric scene representation (5D = xyz + direction)
- ▶ Use volume rendering model to synthesize new views
- ▶ Optimize using rendering loss for one scene (no prior training)
- ▶ One extra trick for passing coordinates into network to get high frequency details

Representing a scene as a continuous 5D function



Neural network replaces large N-d array

14

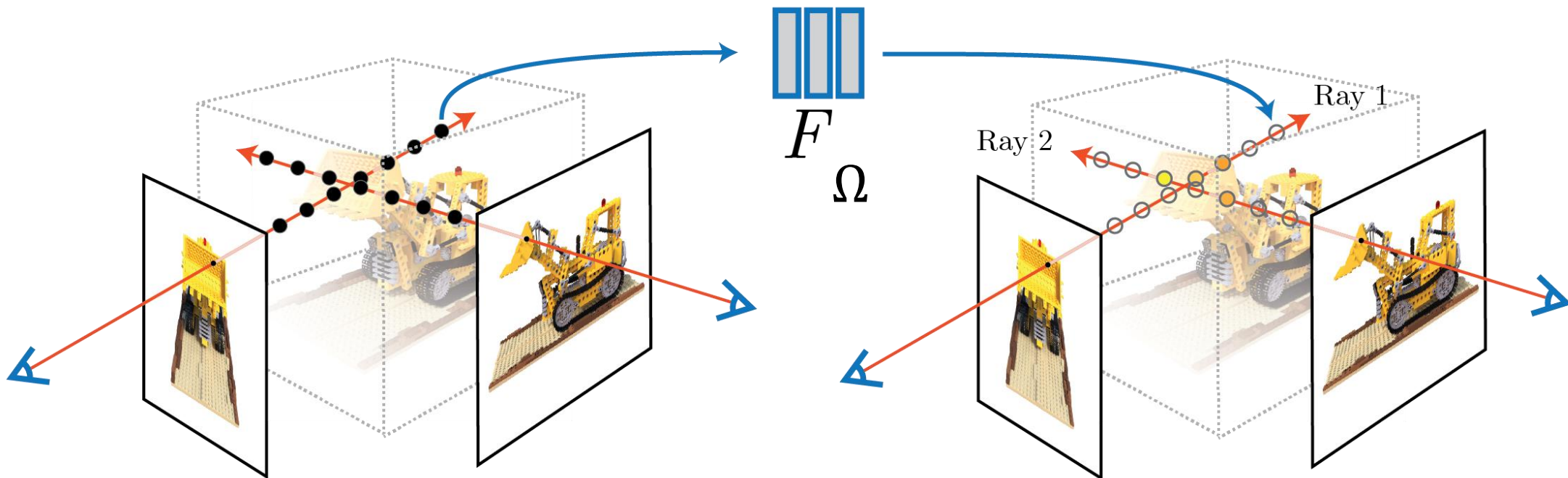


Key points

- ▶ Continuous neural network as a volumetric scene representation (5D = xyz + direction)
- ▶ **Use volume rendering model to synthesize new views**
- ▶ Optimize using rendering loss for one scene (no prior training)
- ▶ One extra trick for passing coordinates into network to get high frequency details

Generate views with traditional volume rendering

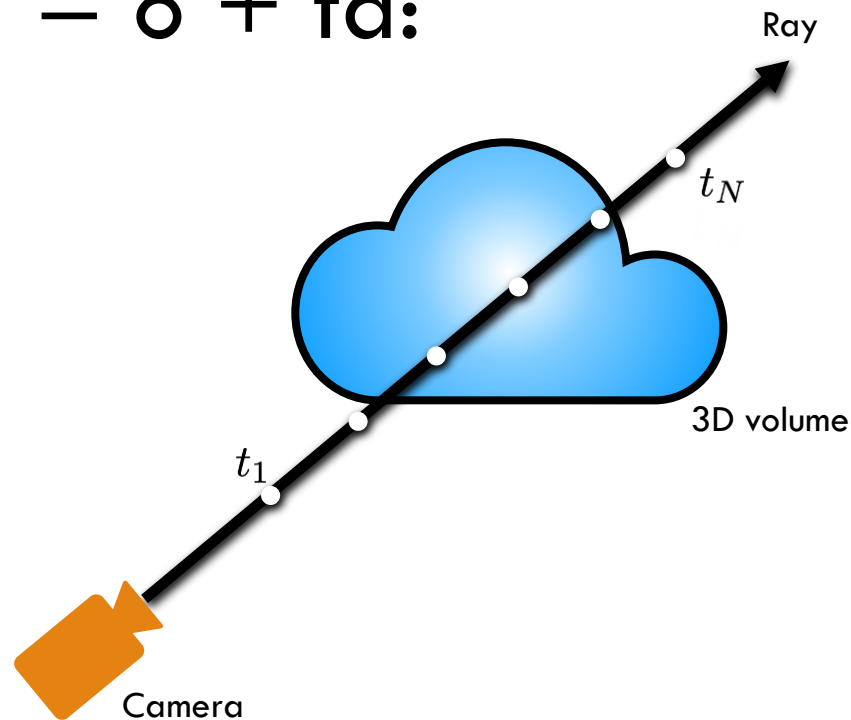
16



Generate views with traditional volume rendering

17

Rendering model for ray $r(t) = o + td$:

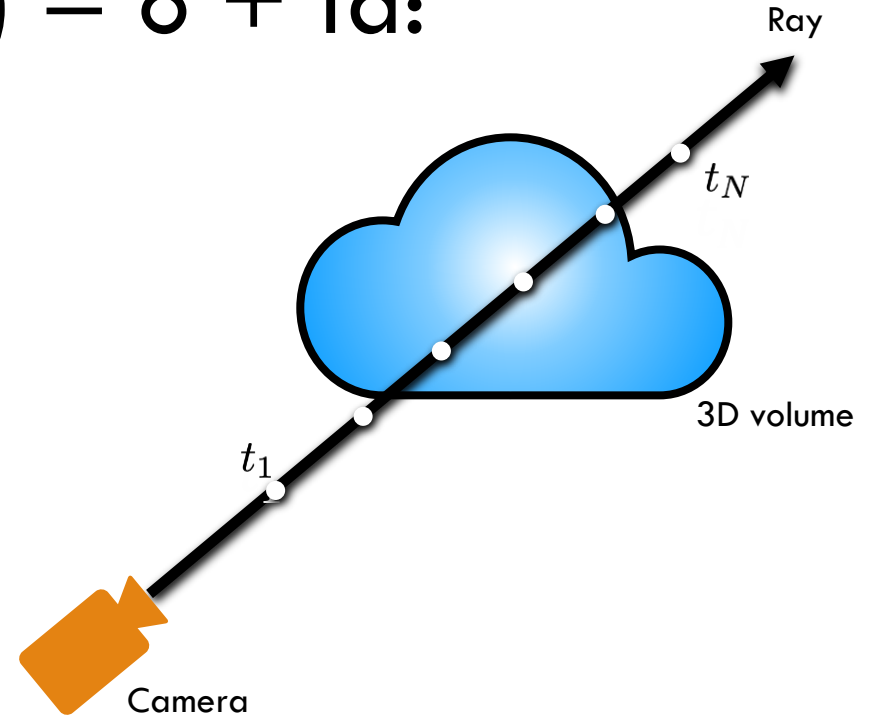


Generate views with traditional volume rendering

Rendering model for ray $r(t) = o + td$:

$$C \approx \sum_{i=1}^N T_i \alpha_i c_i$$

weights colors



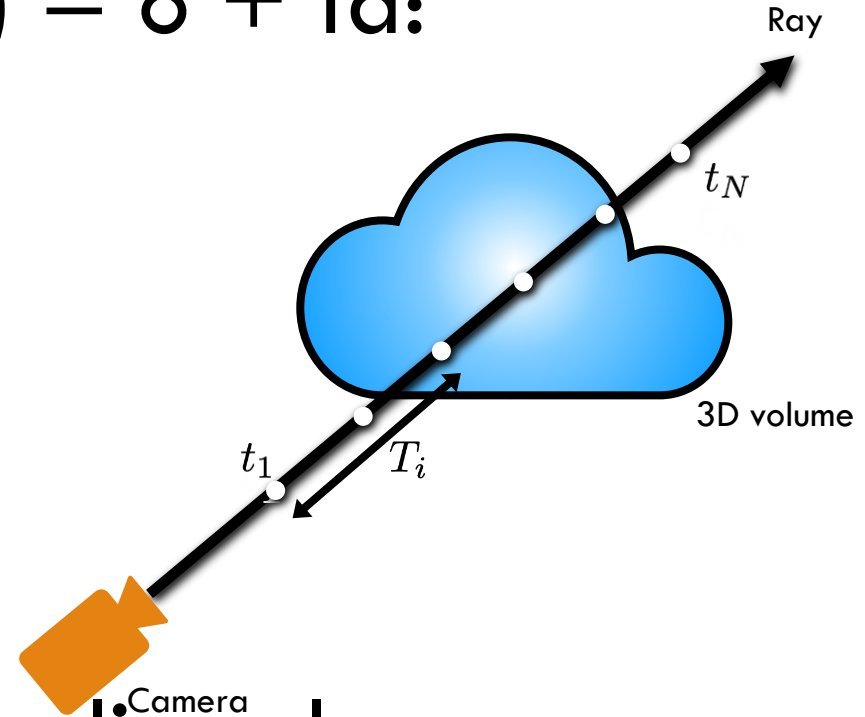
Generate views with traditional volume rendering

Rendering model for ray $r(t) = o + td$:

$$C \approx \sum_{i=1}^N T_i \alpha_i c_i$$

↑ weights
↑ colors

$$T_i = \prod_{j=1}^{i-1} (1 - \alpha_j)$$



How much light is blocked earlier along ray:

Generate views with traditional volume rendering

Rendering model for ray $r(t) = o + td$:

$$C \approx \sum_{i=1}^N T_i \alpha_i c_i$$

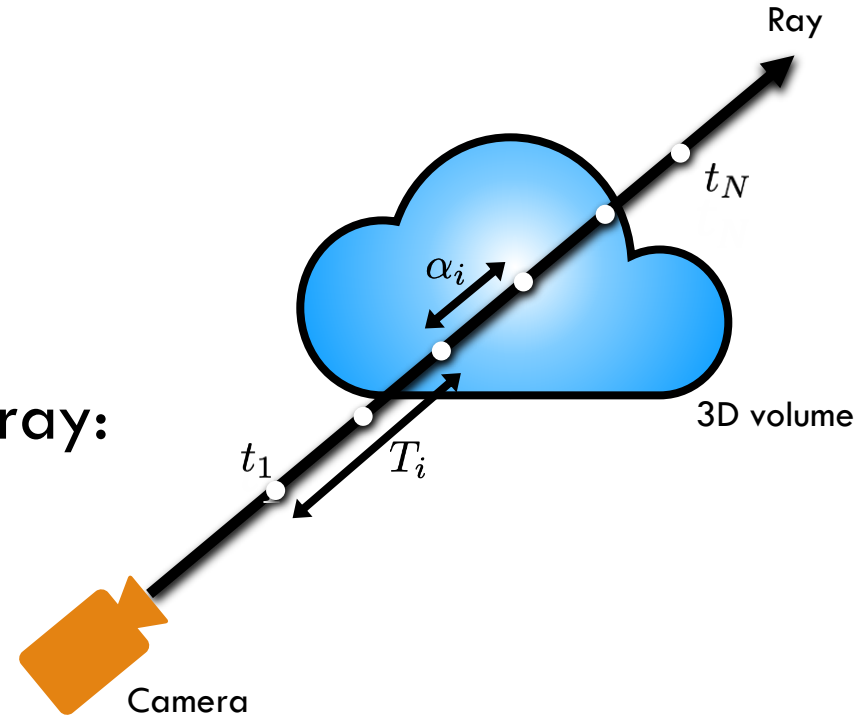
↑ weights
↑ colors

How much light is blocked earlier along ray:

$$T_i = \prod_{j=1}^{i-1} (1 - \alpha_j)$$

How much light is contributed by ray segment i :

$$\alpha_i = 1 - e^{-\sigma_i \delta t_i}$$



Sigma parametrization for continuous opacity

Rendering model for ray $r(t) = o + td$:

$$C \approx \sum_{i=1}^N T_i \alpha_i c_i$$

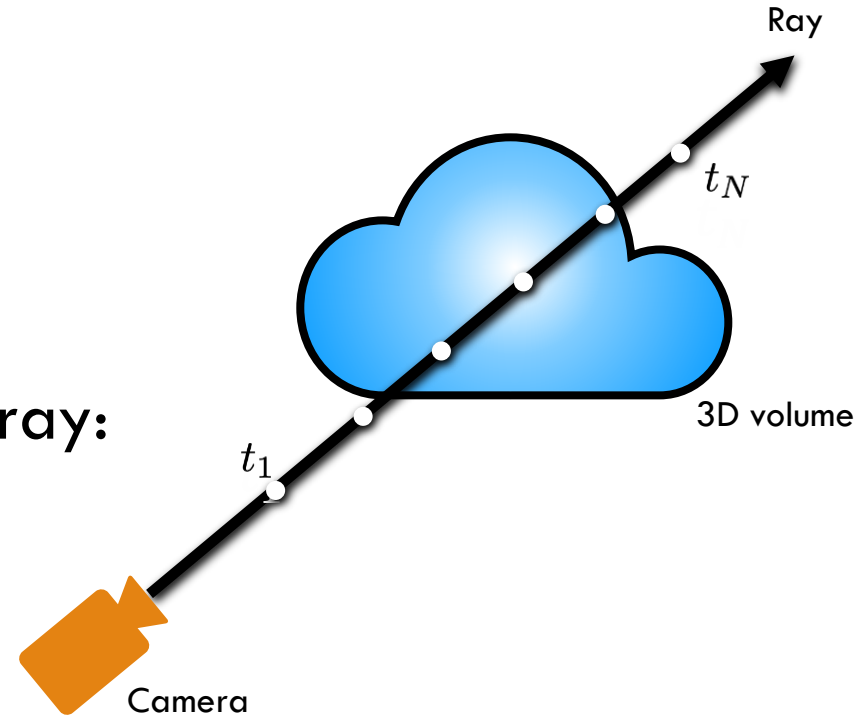
↑ weights
↑ colors

How much light is blocked earlier along ray:

$$T_i = \prod_{j=1}^{i-1} (1 - \alpha_j)$$

How much light is contributed by ray segment i :

$$\alpha_i = 1 - e^{-\sigma_i \delta t_i}$$



Effective resolution is tied to distance between samples

Rendering model for ray $r(t) = o + td$:

$$C \approx \sum_{i=1}^N T_i \alpha_i c_i$$

↑ weights
 ↑ colors

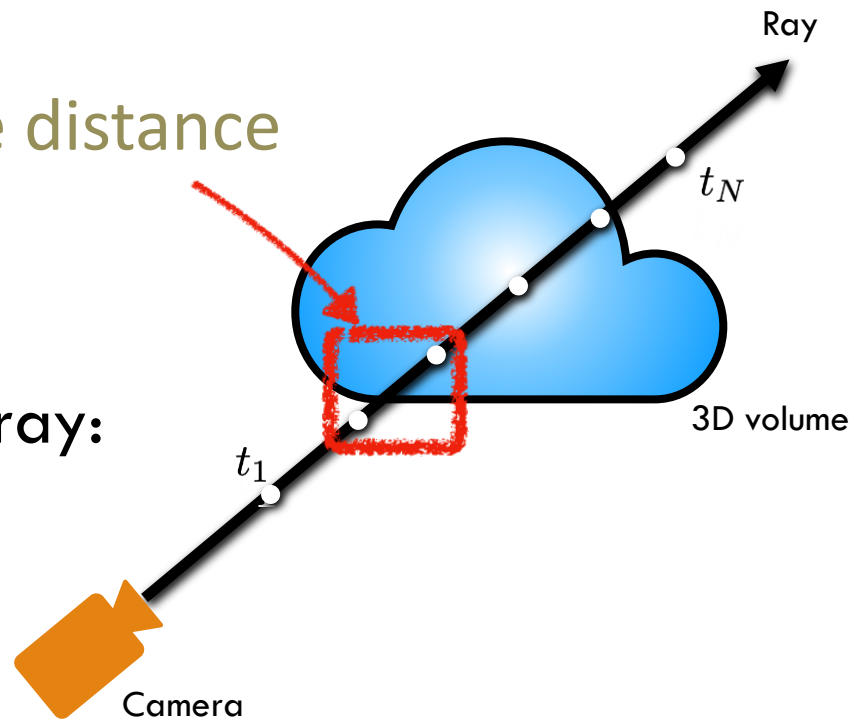
sample distance

How much light is blocked earlier along ray:

$$T_i = \prod_{j=1}^{i-1} (1 - \alpha_j)$$

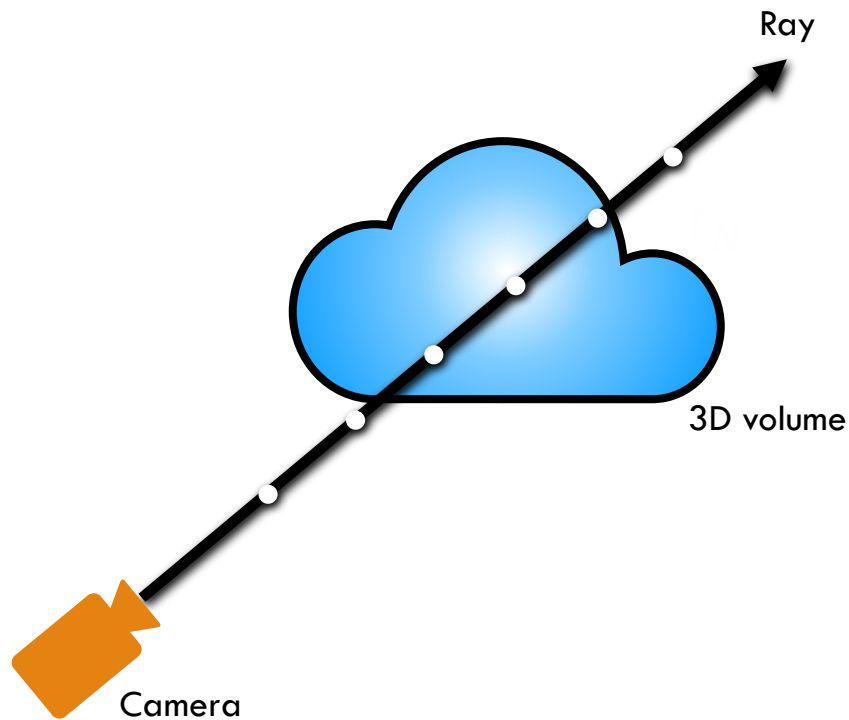
How much light is contributed by ray segment i :

$$\alpha_i = 1 - e^{-\sigma_i \delta t_i}$$



Can we allocate samples more efficiently? Two pass rendering

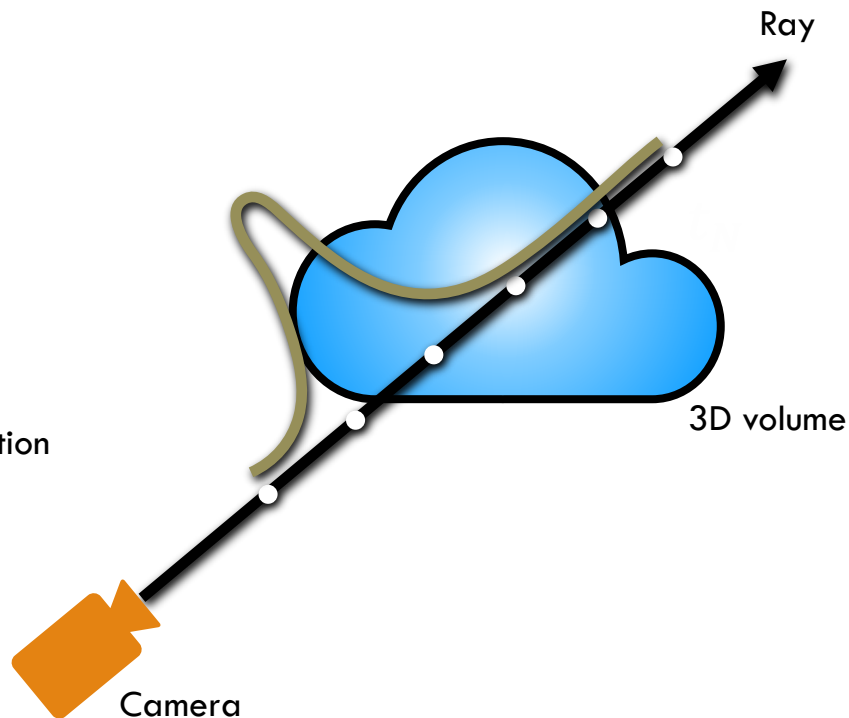
23



Two pass rendering: coarse

$$C \approx \sum_{i=1}^N T_i \alpha_i c_i$$

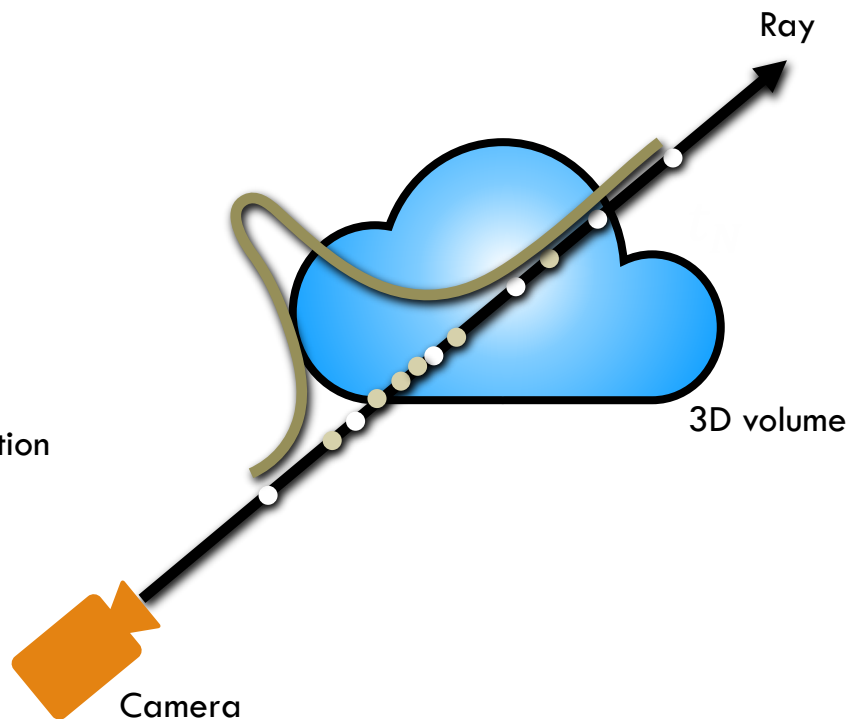
treat weights as probability distribution
for new samples



Two pass rendering: fine

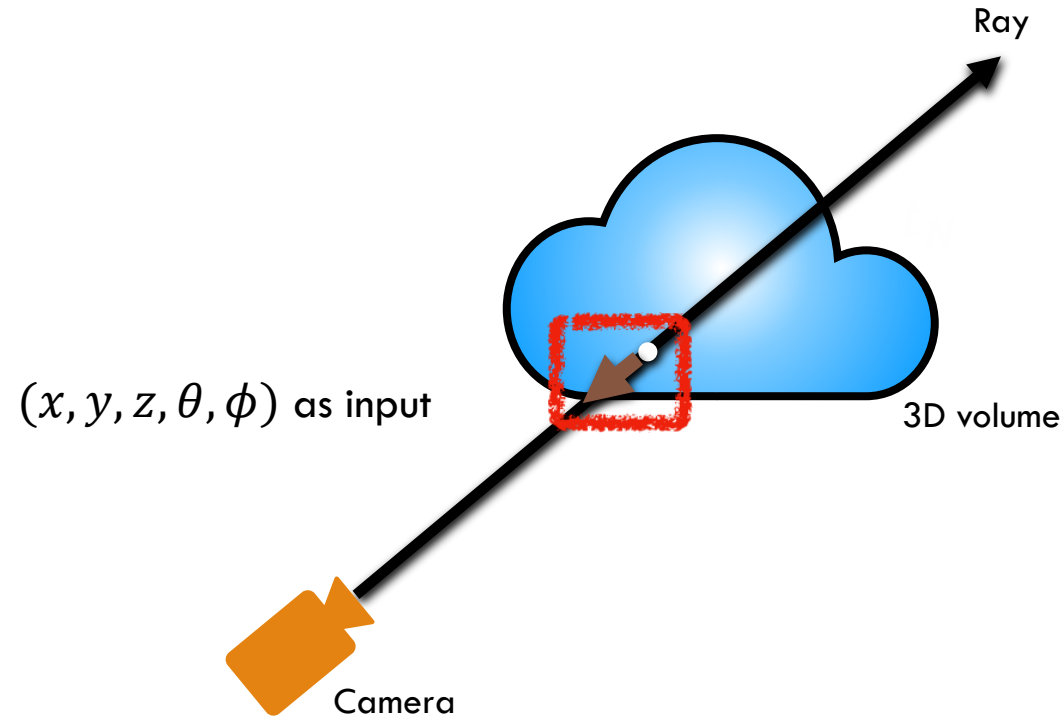
$$C \approx \sum_{i=1}^N T_i \alpha_i c_i$$

treat weights as probability distribution
for new samples



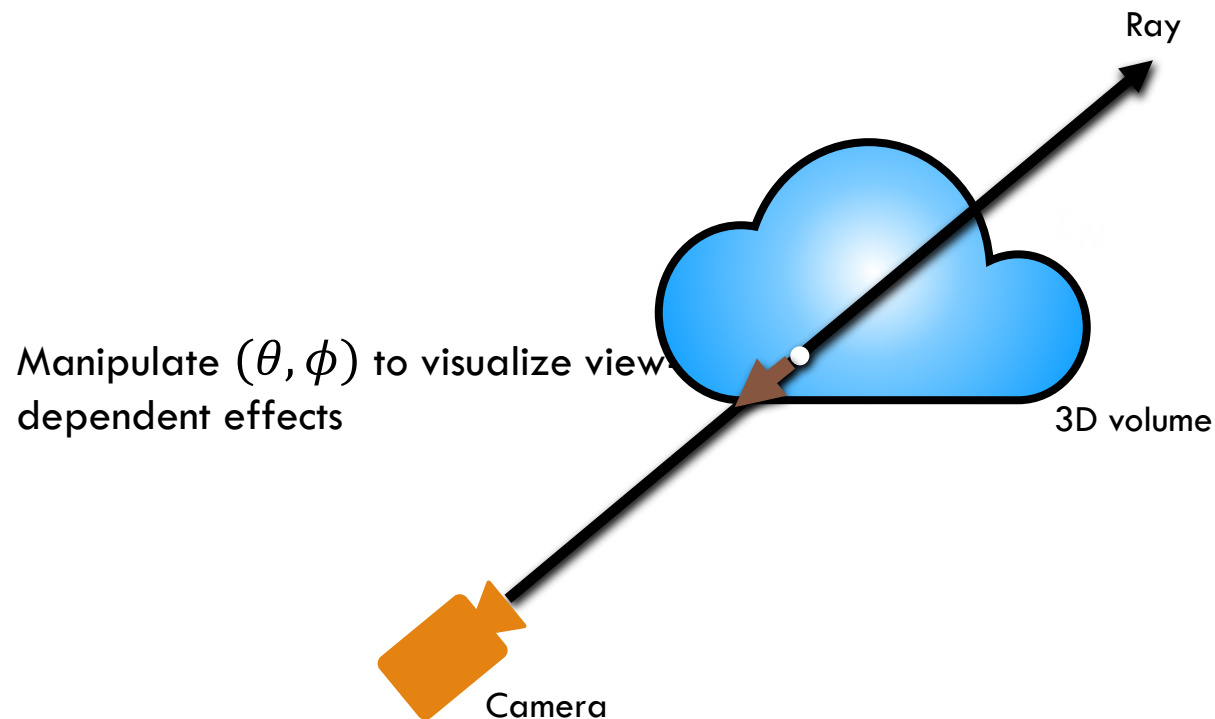
Viewing directions as input

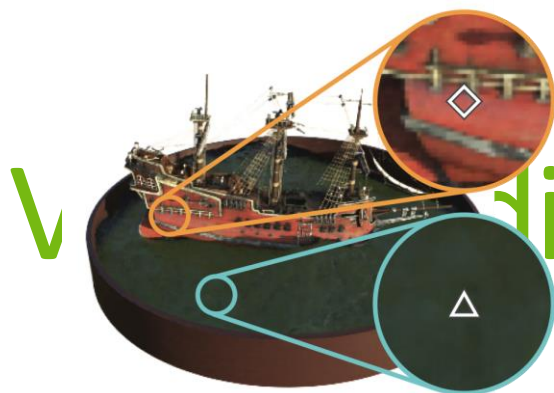
26



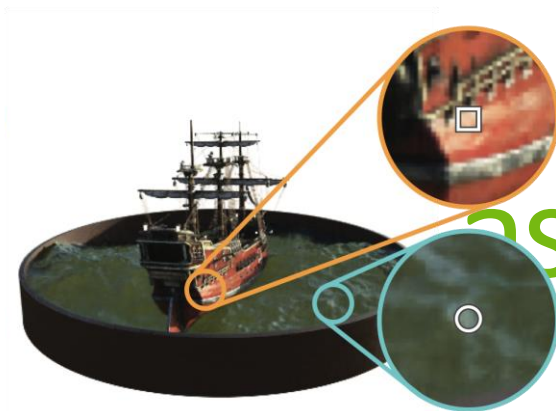
Viewing directions as input

27

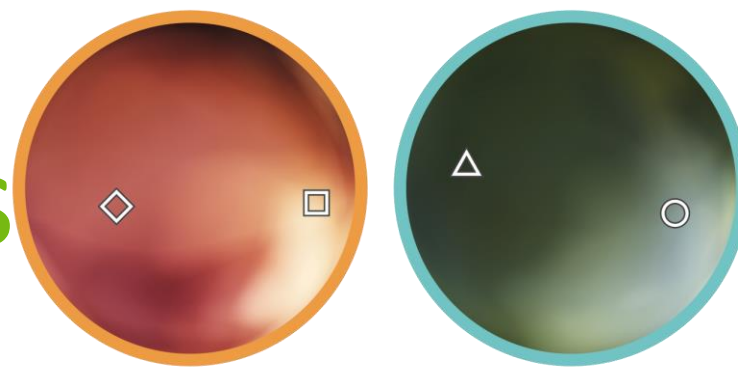




(a) View 1



(b) View 2

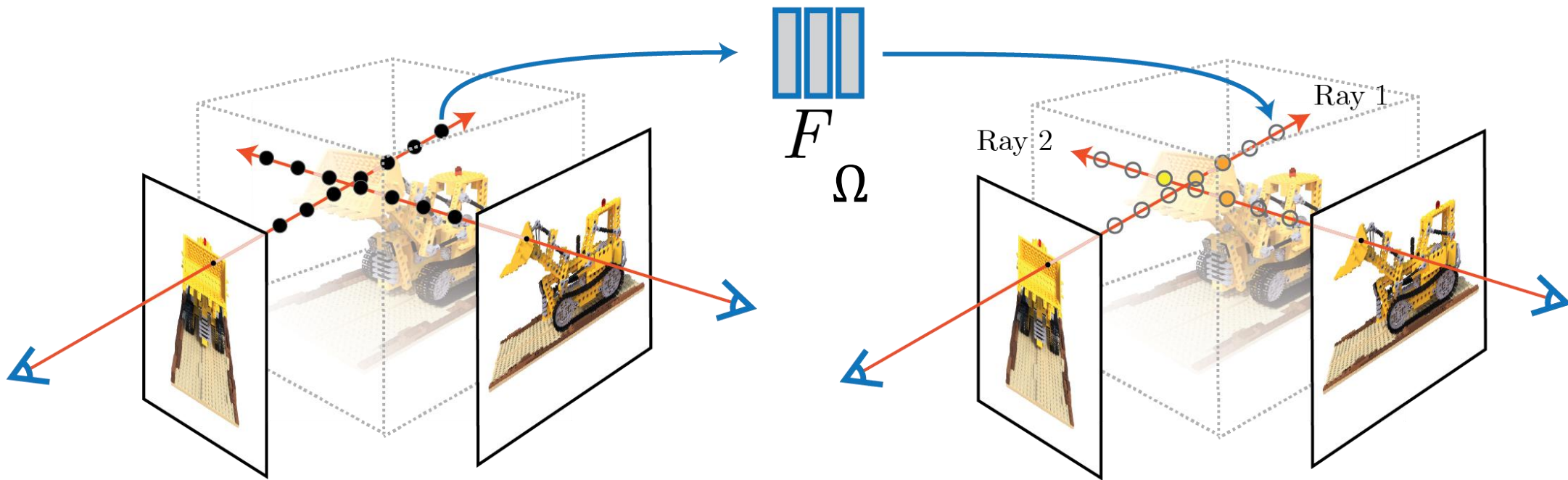


(c) Radiance Distributions

Key points

- ▶ Continuous neural network as a volumetric scene representation (5D = xyz + direction)
- ▶ Use volume rendering model to synthesize new views
- ▶ **Optimize using rendering loss for one scene (no prior training)**
- ▶ One extra trick for passing coordinates into network to get high frequency details

Optimize with gradient descent on rendering loss



$$\min_{\Omega} \sum_i \| \text{render}^{(i)}(F_{\Omega}) - I_{\text{gt}}^{(i)} \|^2$$

Training network to reproduce all input views of the scene

32



Naive implementation produces blurry results

33



NeRF (Naive)

Naive implementation produces blurry results



NeRF (Naive)



NeRF (with positional encoding)

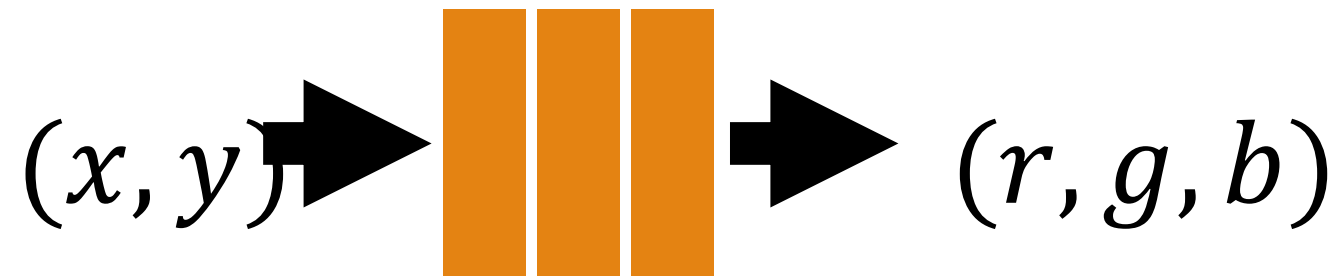
Key points

- ▶ Continuous neural network as a volumetric scene representation (5D = xyz + direction)
- ▶ Use volume rendering model to synthesize new views
- ▶ Optimize using rendering loss for one scene (no prior training)
- ▶ **One extra trick for passing coordinates into network to get high frequency details**

Challenge:

- ▶ How to get MLPs to represent higher frequency functions?

Simpler toy problem: memorizing a 2D image

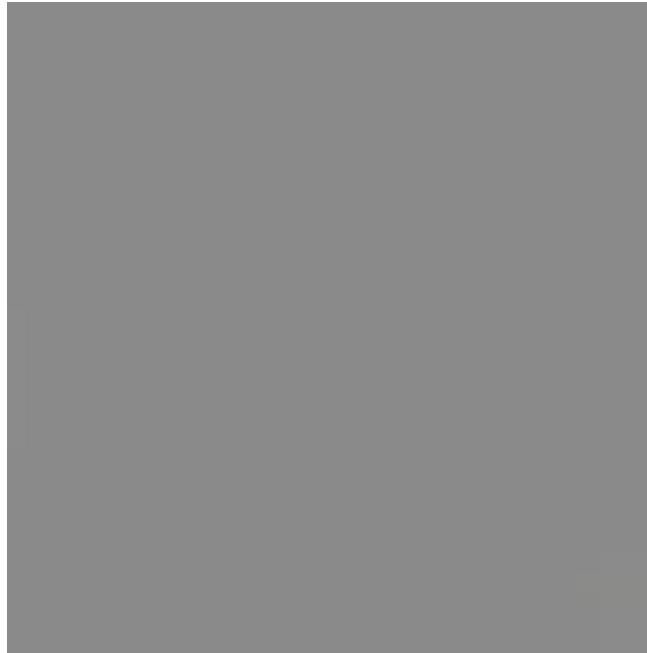


Simple trick enables network to memorize images

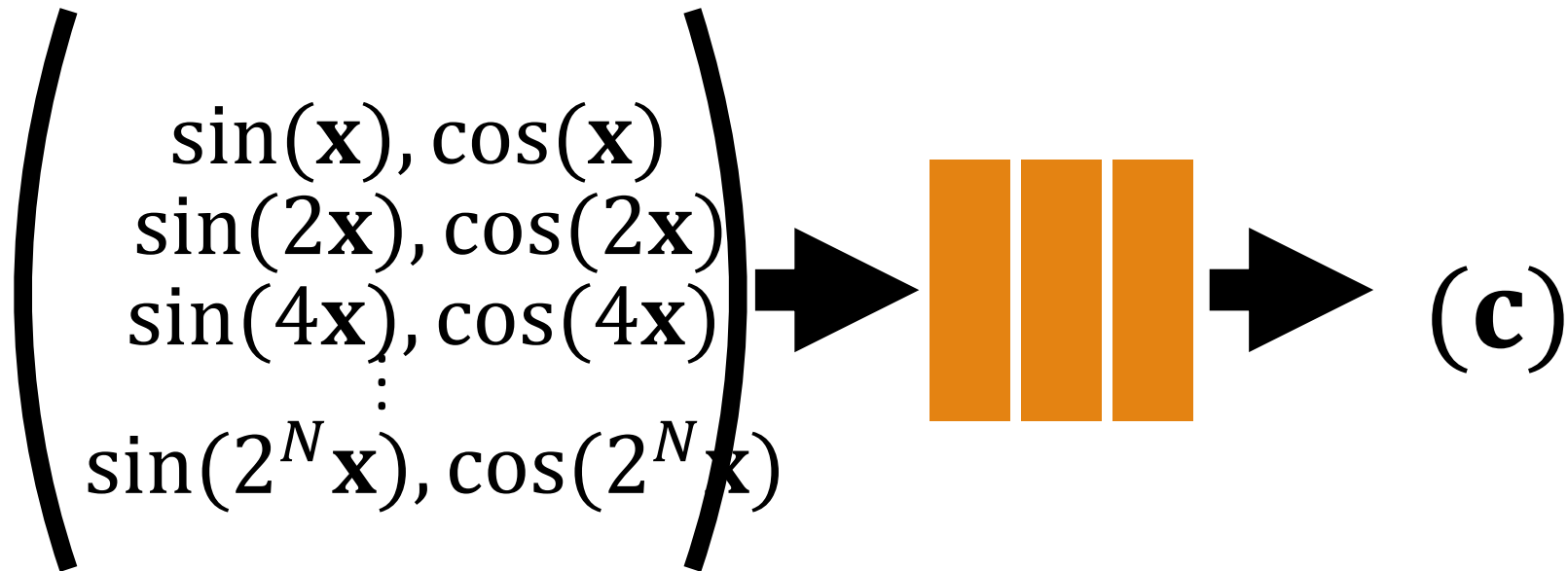
Ground truth image



Standard fully-connected net



Positional encoding: high frequency embedding of input coordinates



Simple trick enables network to memorize images

Ground truth image



Standard fully-connected net



With “embedding”



Positional encoding also directly improves our scene representation!

41



NeRF (Naive)



NeRF (with positional encoding)

Results





44

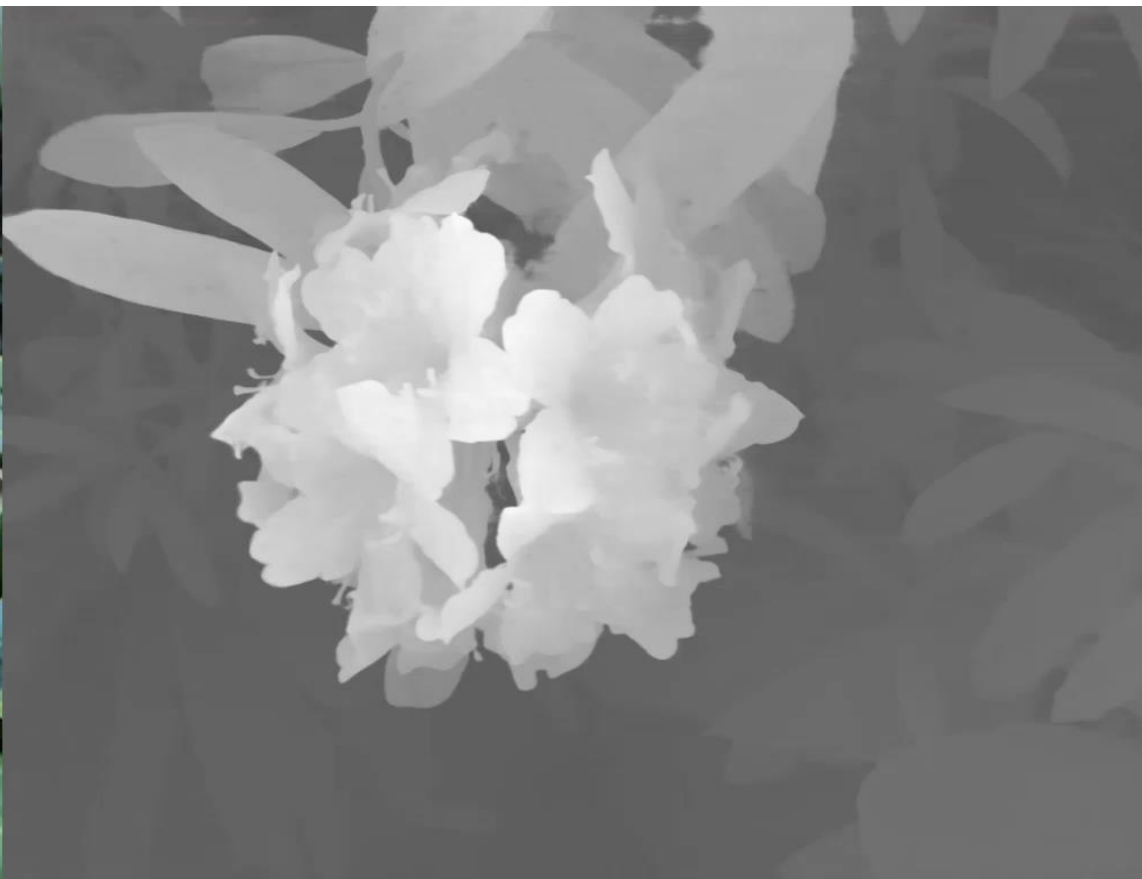
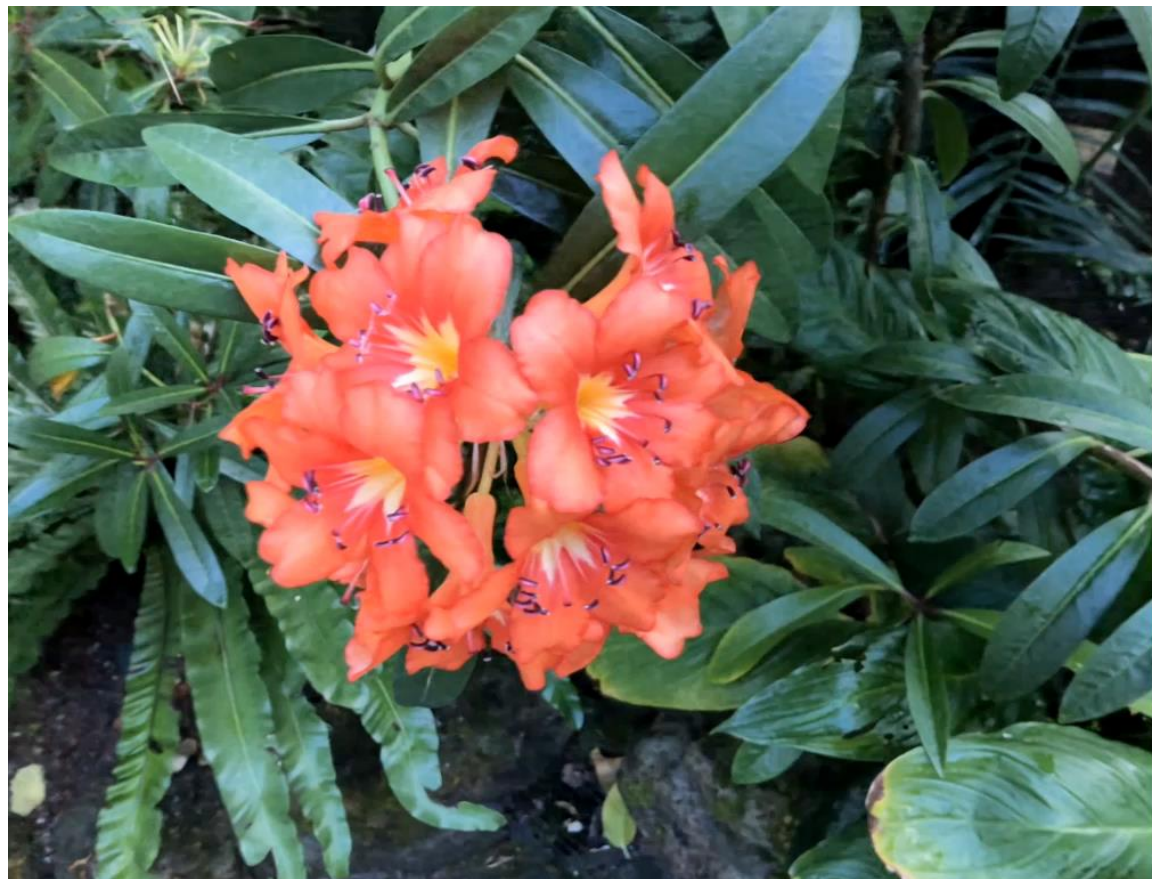




NeRF encodes convincing view-dependent effects using directional dependence

46





NeRF encodes detailed scene geometry with occlusion effects

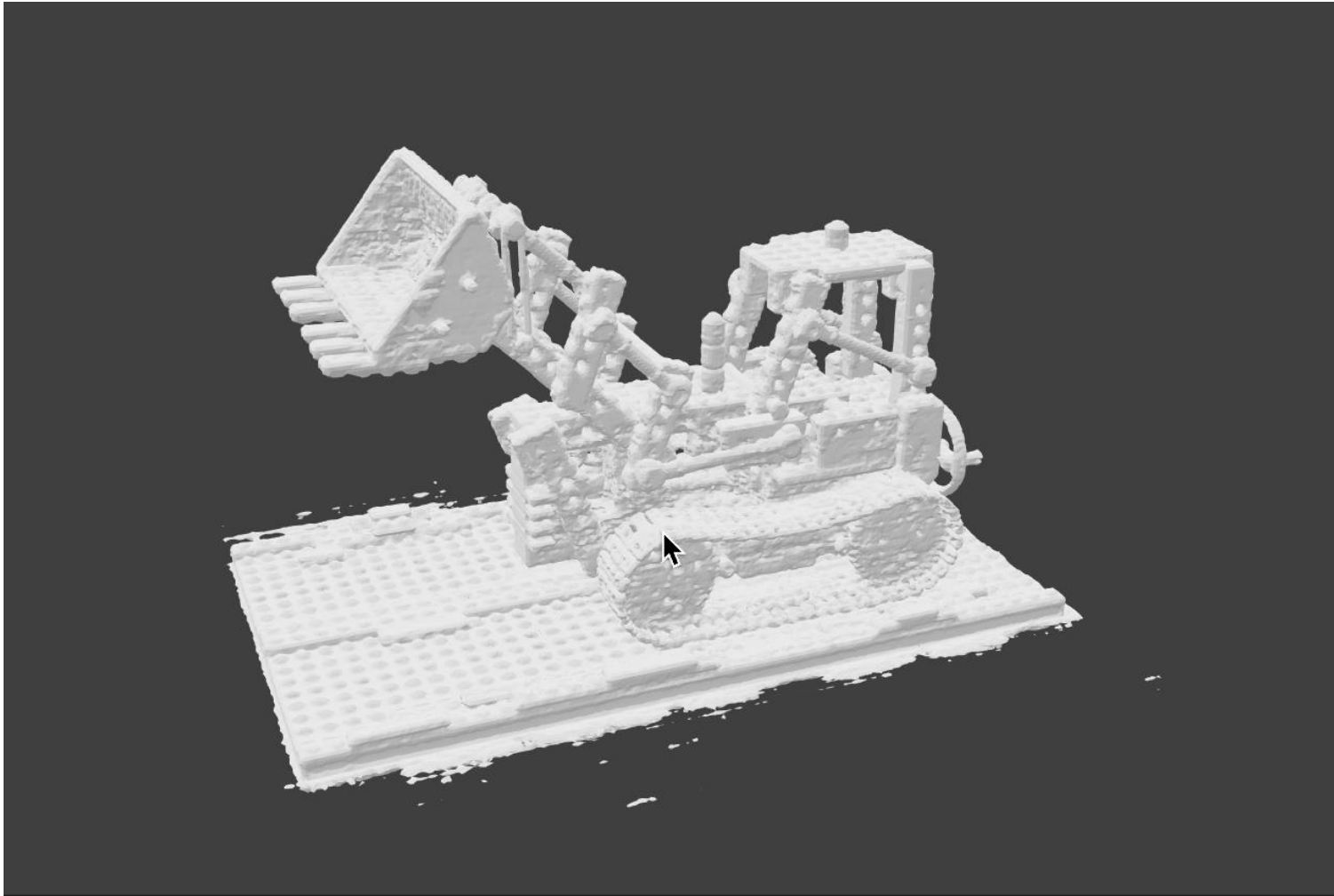


NeRF encodes detailed scene geometry with occlusion effects



NeRF encodes detailed scene geometry

50



Block-NeRF: Scalable Large Scene Neural View Synthesis

Matthew Tancik^{1*} Vincent Casser² Xincheng Yan² Sabeek Pradhan²
Ben Mildenhall³ Pratul P. Srinivasan³ Jonathan T. Barron³ Henrik Kretzschmar²
¹UC Berkeley ²Waymo ³Google Research



Reconstructions of San Francisco

4. Method

4.1. Block Size and Placement

The individual Block-NeRFs should be arranged to collectively ensure full coverage of the target environment.

- ❑ Place one Block-NeRF at each intersection

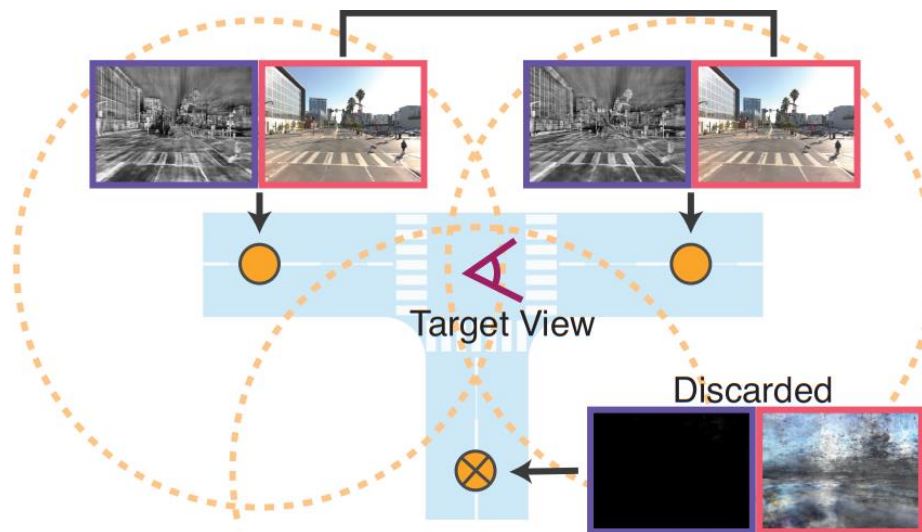
The block size is variable



covering the intersection itself
and any connected street 75% of the way until it
converges into the next intersection
⇒ 50% overlap between any two adjacent
making appearance alignment easier

- ❑ Place blocks along a single street segment at uniform distances

Define the block size as a sphere around the Block-NeRF Origin



- Additional blocks may be introduced as connectors between intersections
- Using geographical filter: ensure that the training data for each block stays exactly within its intended bounds

5. Results and Experiments

BLOCK-NeRF

Supplemental Results

- + Visibility Prediction Visualizations
- + Appearance Embedding Visualizations
- + Block-NeRF Ablations
- + Comparison of Overlap Percentage
- + Comparison of Interpolation Methods

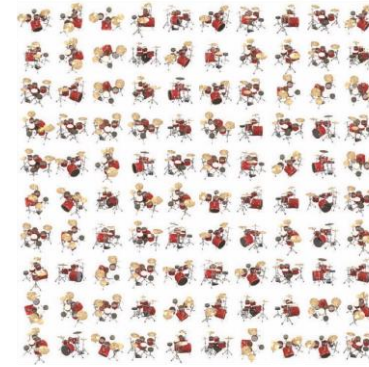
Rendering (Graphics): Given 3D Scene + Camera parameters, yield images



3D Scene

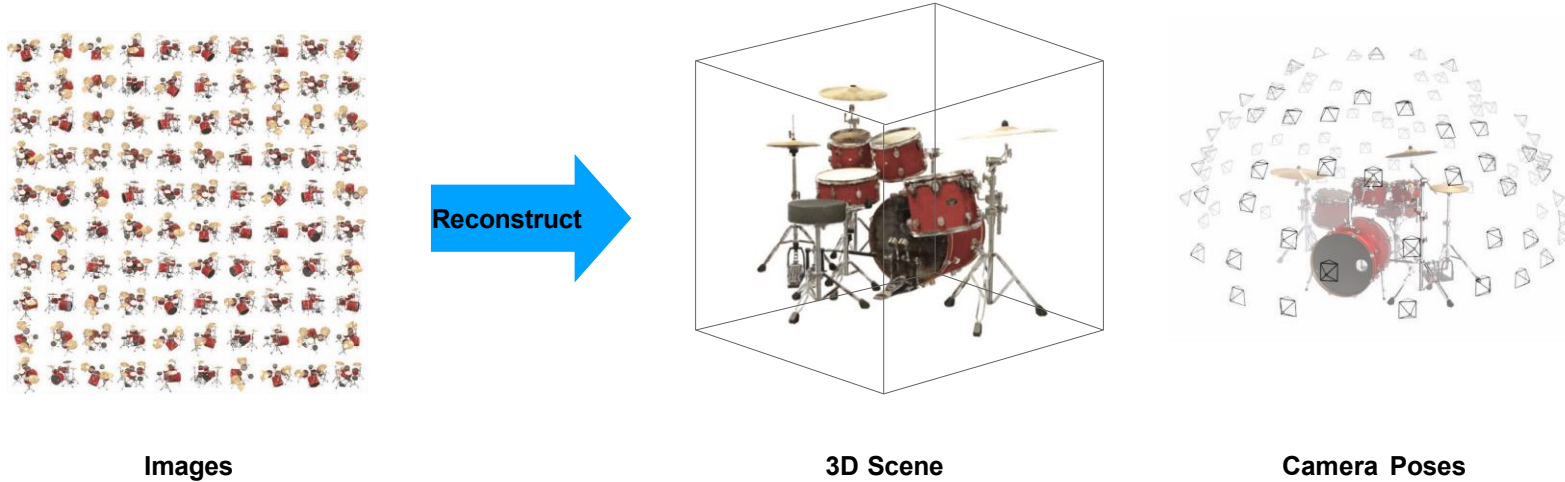


Camera Poses



Images

Inverse Graphics: Given Images, Infer Camera Poses & 3D Scene!



How to get camera poses?



Images



3D Scene

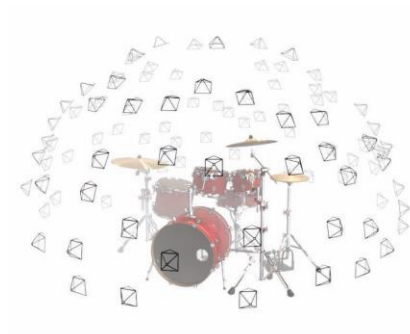


Camera Poses

Can assume we know the camera poses.



Images



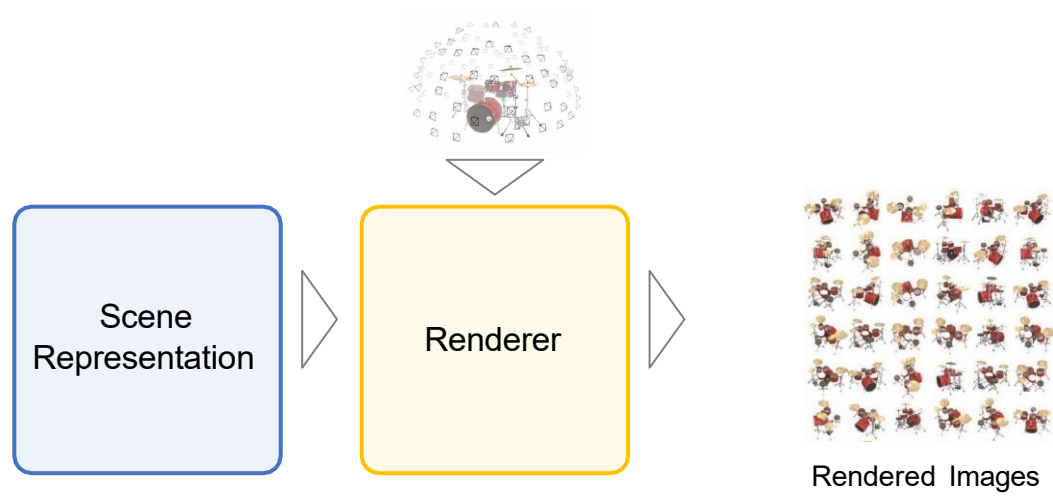
Camera Poses

Reconstruct

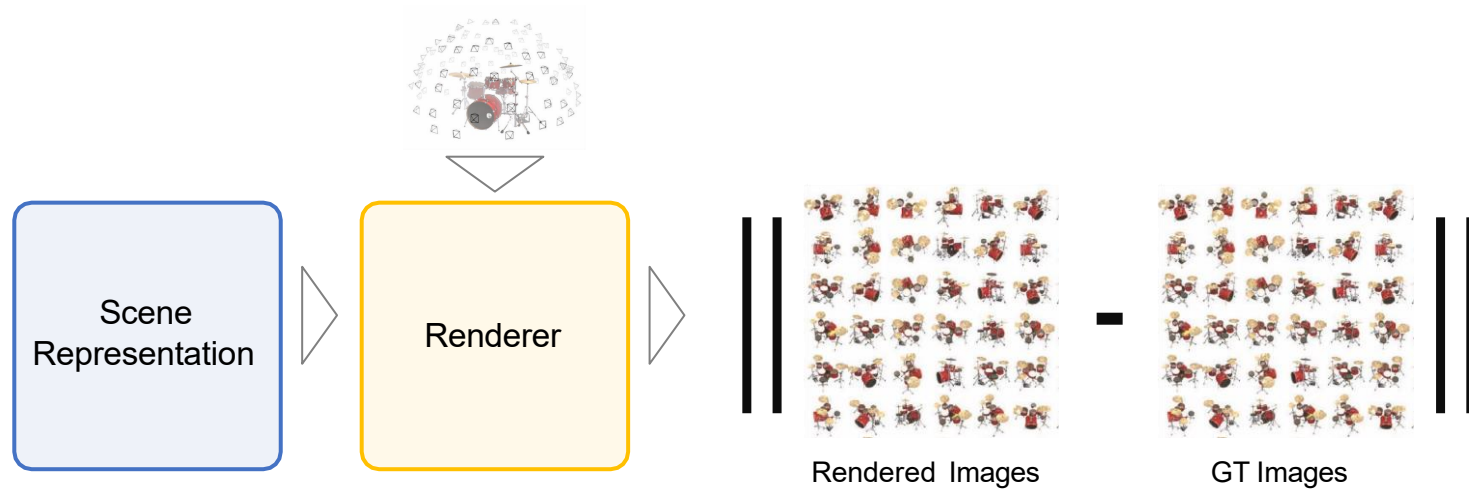


3D Scene

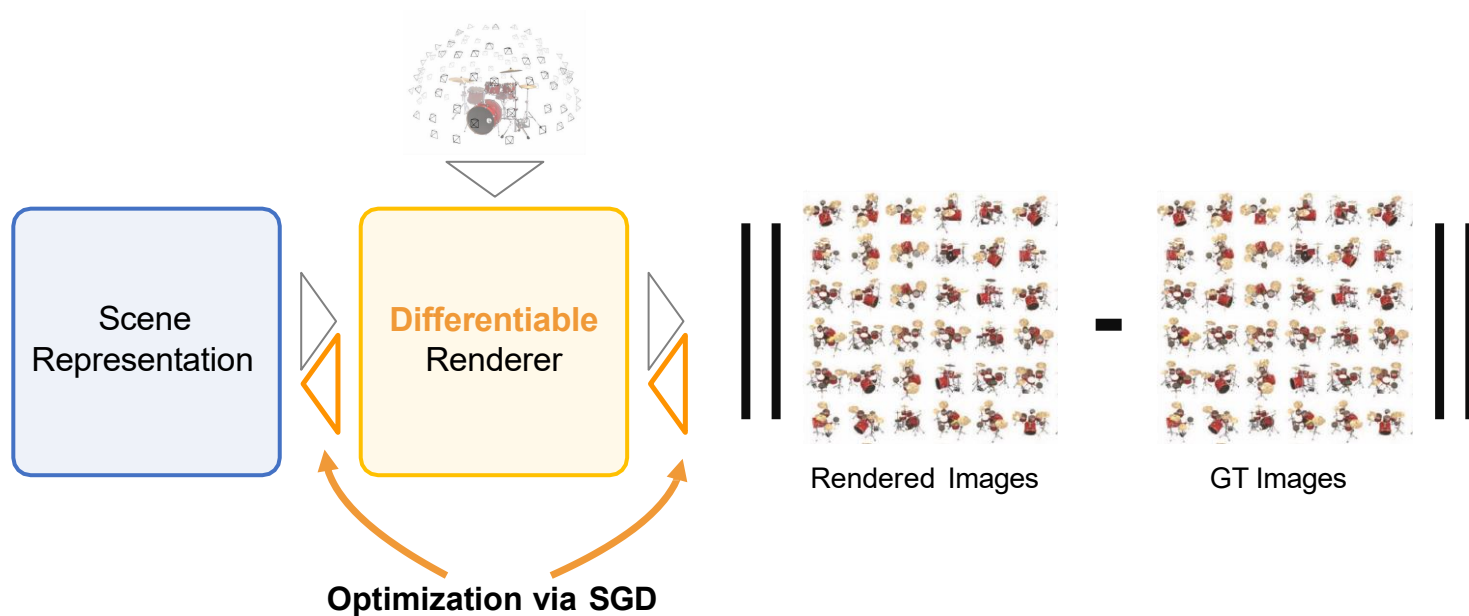
Differentiable Rendering



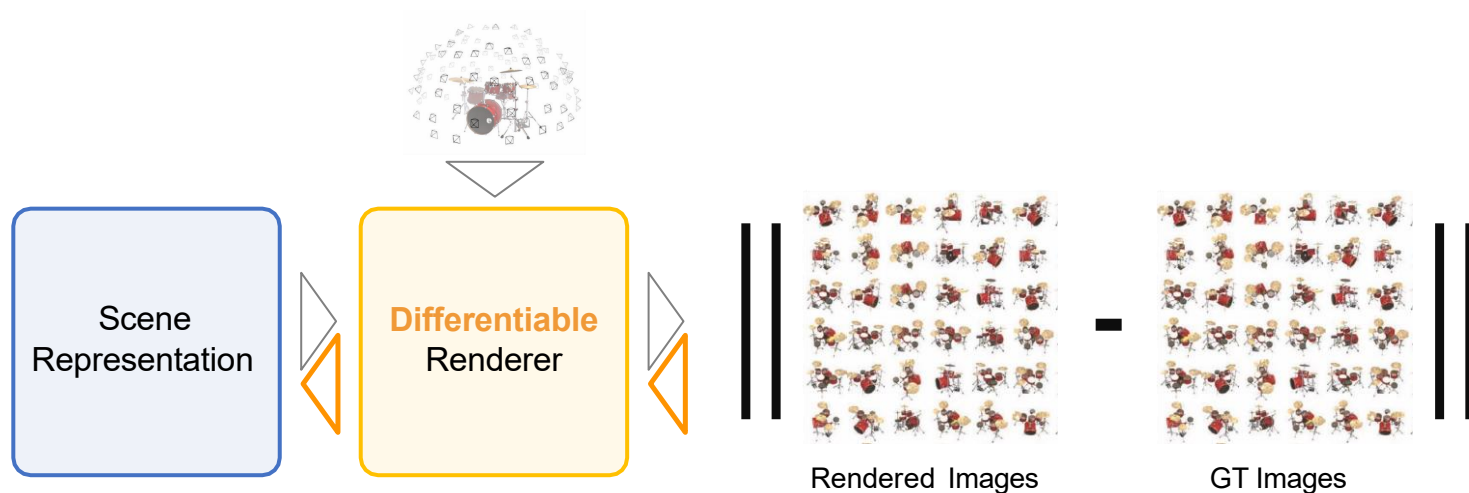
Differentiable Rendering



Differentiable Rendering

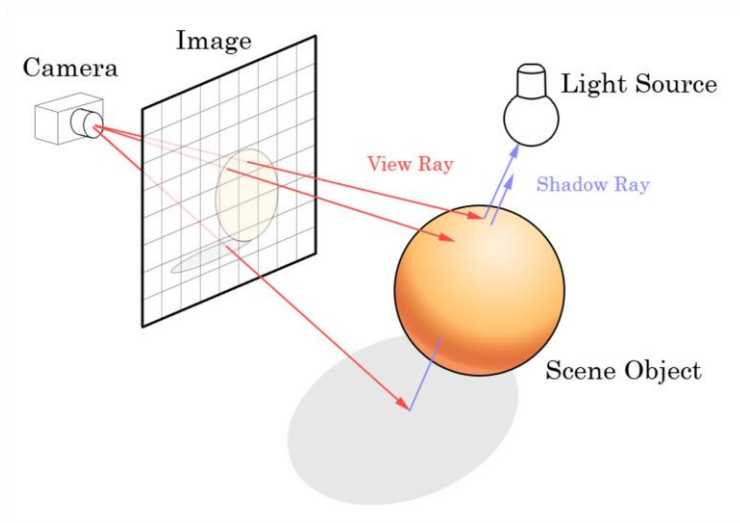


Differentiable Rendering

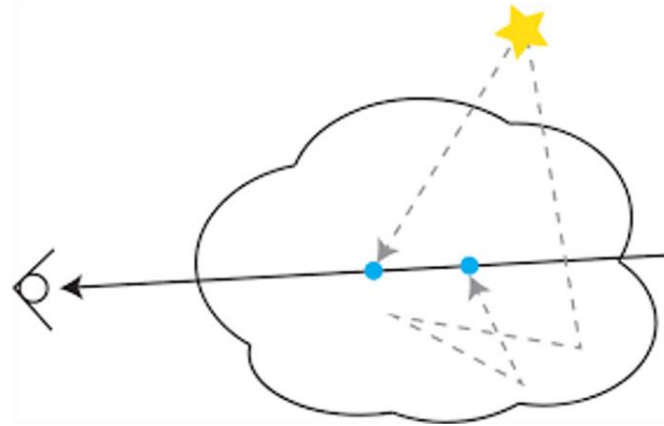


Given an *observable* variable (pixel colors), we will build a differentiable forward model that we then use to estimate *unobserved (latent) variables* (geometry, appearance)!

Ways to Render



Surface rendering



Volume rendering

Volume rendering equation

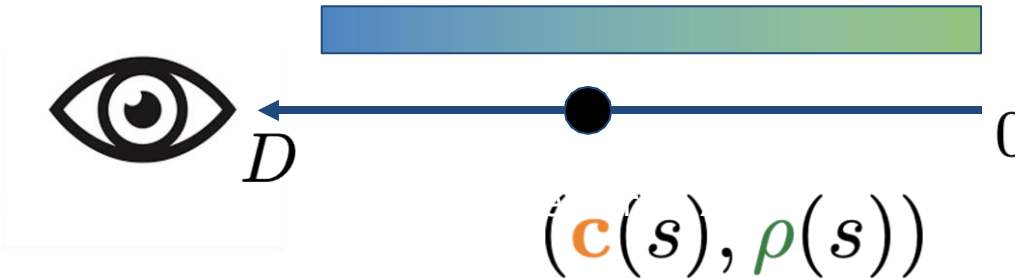
$$\mathbf{I}(D) = \mathbf{I}_0 T(0) + \int_0^D \mathbf{c}(s) \rho(s) T(s) ds$$

pixel color at coordinates D

radiance density

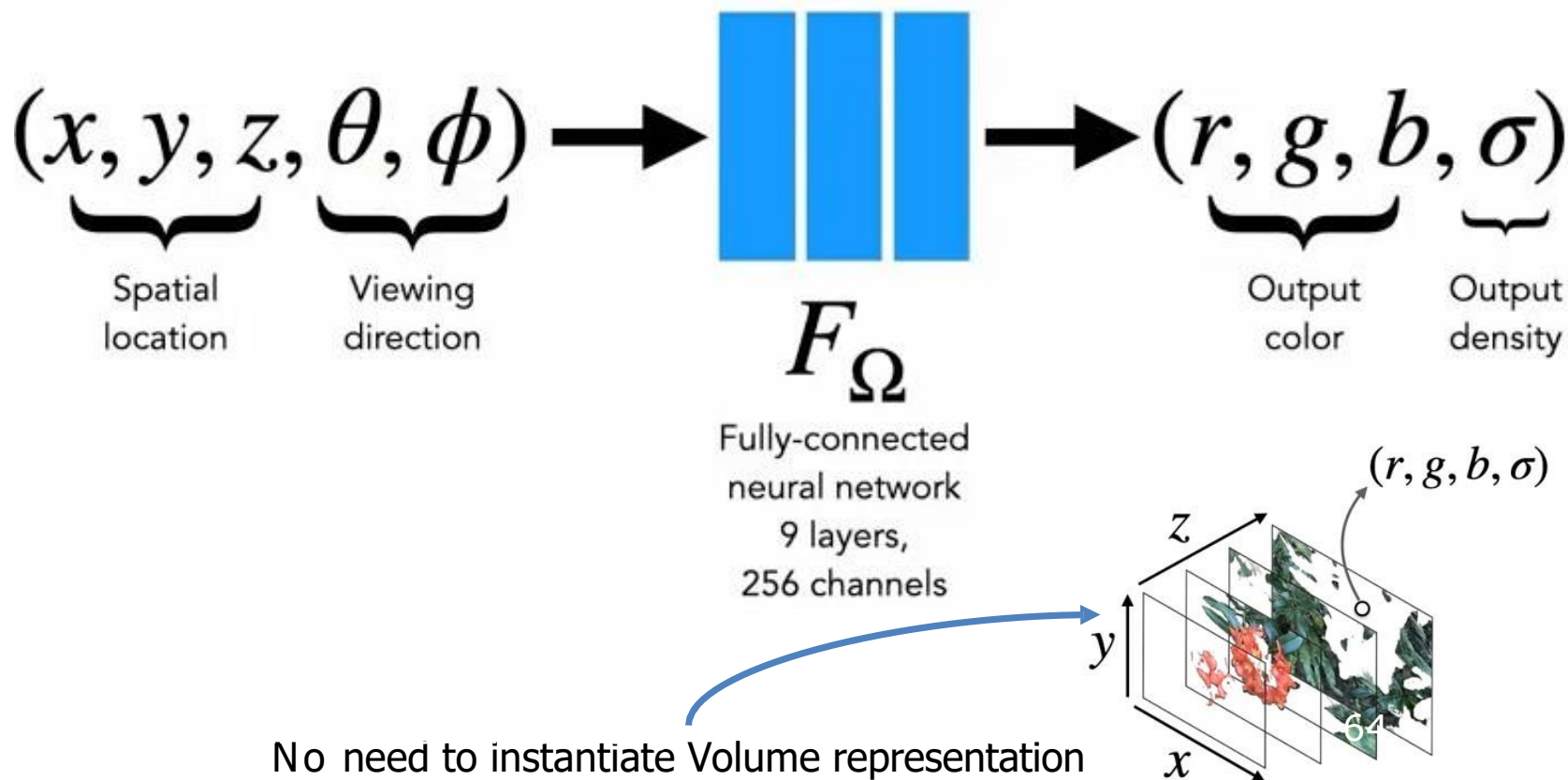
$$T(s) = \exp \left(- \int_s^D \rho(t) dt \right)$$

transparency



The diagram illustrates the volume rendering process. It shows an eye icon labeled D on the left, looking towards a horizontal line segment from 0 to D . A point on the line is labeled $(\mathbf{c}(s), \rho(s))$. Above the line is a color gradient bar transitioning from blue to green, representing the radiance density $\rho(s)$ and color $\mathbf{c}(s)$ along the path.

Represent a scene as a continuous 5D function



Generate views with traditional volume rendering

Rendering model for ray $r(t) = o + td$:

$$C \approx \sum_{i=1}^N T_i \alpha_i c_i$$

weights colors

t = point along ray
 C = Color of Pixel
 c = color of point

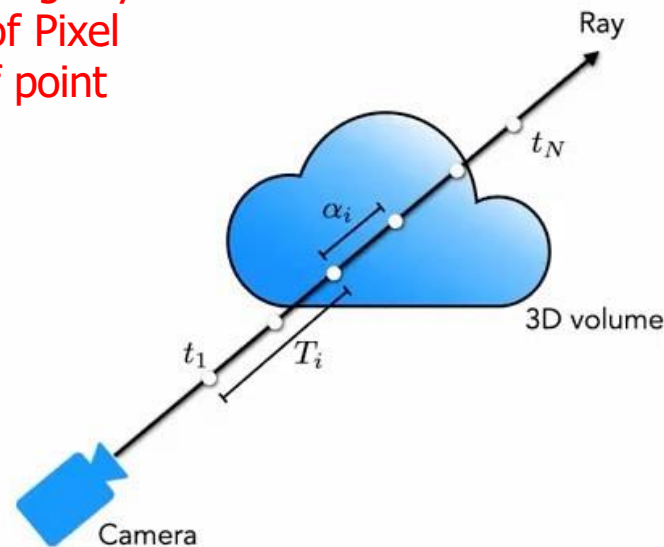
How much light is blocked earlier along ray:

$$T_i = \prod_{j=1}^{i-1} (1 - \alpha_j) \quad \text{Transparency}$$

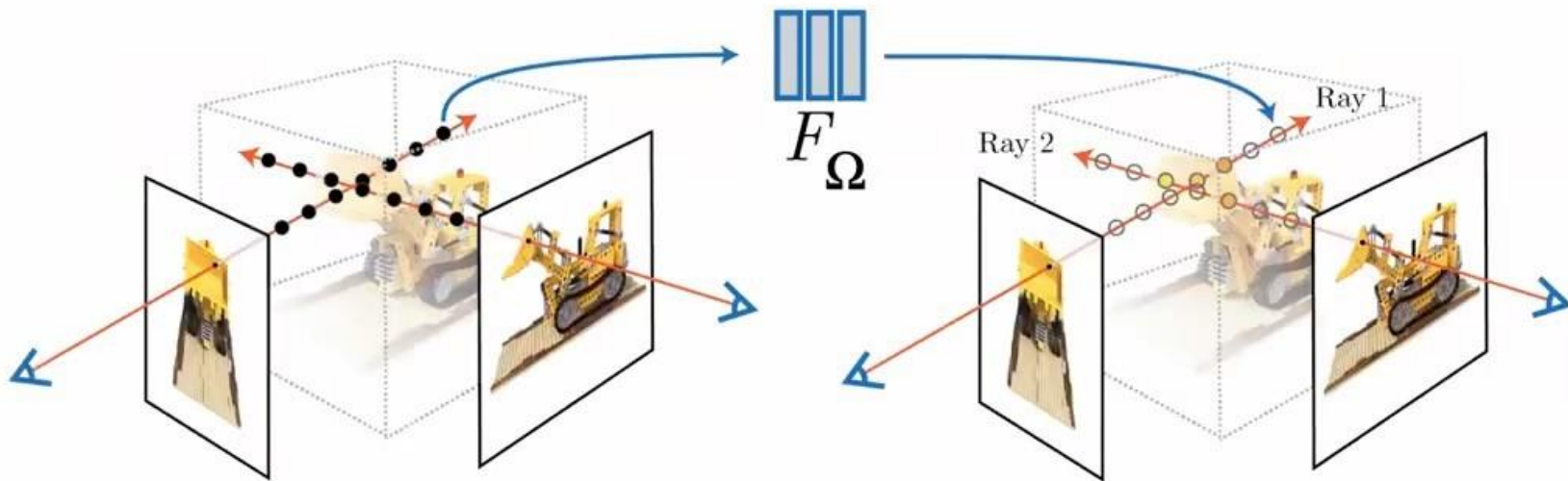
How much light is contributed by ray segment i :

$$\alpha_i = 1 - e^{-\sigma_i \delta t_i}$$

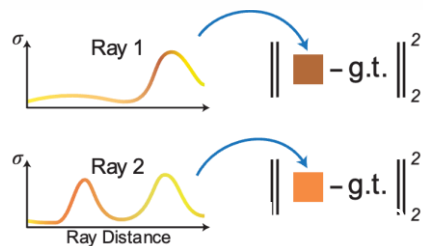
Function of segment length δt_i and volume density σ



Optimize with gradient descent on rendering loss



$$\min_{\Omega} \sum_i \|\text{render}^{(i)}(F_{\Omega}) - I_{\text{gt}}^{(i)}\|^2$$

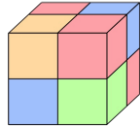


From Presentation by Matthew Tancik: Neural Radiance Fields for View Synthesis. 2020.

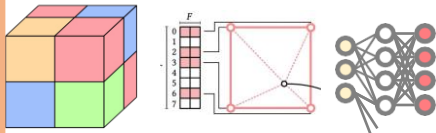
Training network to reproduce all input views of the scene



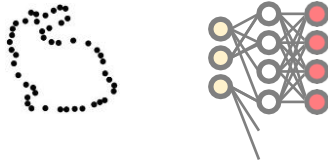
From Presentation by Matthew Tancik: Neural Radiance Fields for View Synthesis. 2020.



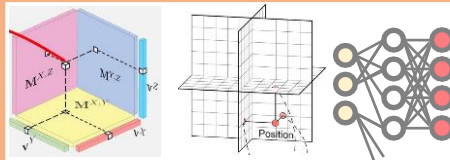
Voxel Grid



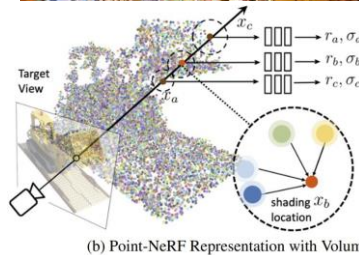
Voxel Grid + Hashmap + MLP



Point Cloud + MLP



Tensor Factorization / Triplane + MLP



Plenoxels: Radiance Fields ...

[Yu et al. 2022]

Direct Voxel Grid Optimization

[Sun et al. 2021]

InstantNGP: Instant Neural ...

[Müller et al. 2022]

PointNeRF: Point-based Neural ... [Xu et al. 2022]

Efficient Geometry-aware 3D... [Chan et al. 2022]

TensorRF: Tensor Radiance Fields [Chen & Xu et al. 2022]

Hybrid Multi-Scale Grid,
HashMap, Neural Field

Instant-NGP

9.0

0.07

MipNeRF360

Neural Field

FPS

3D Gaussian Splatting for Real-Time Radiance Field Rendering

BERNHARD KERBL*, Inria, Université Côte d’Azur, France

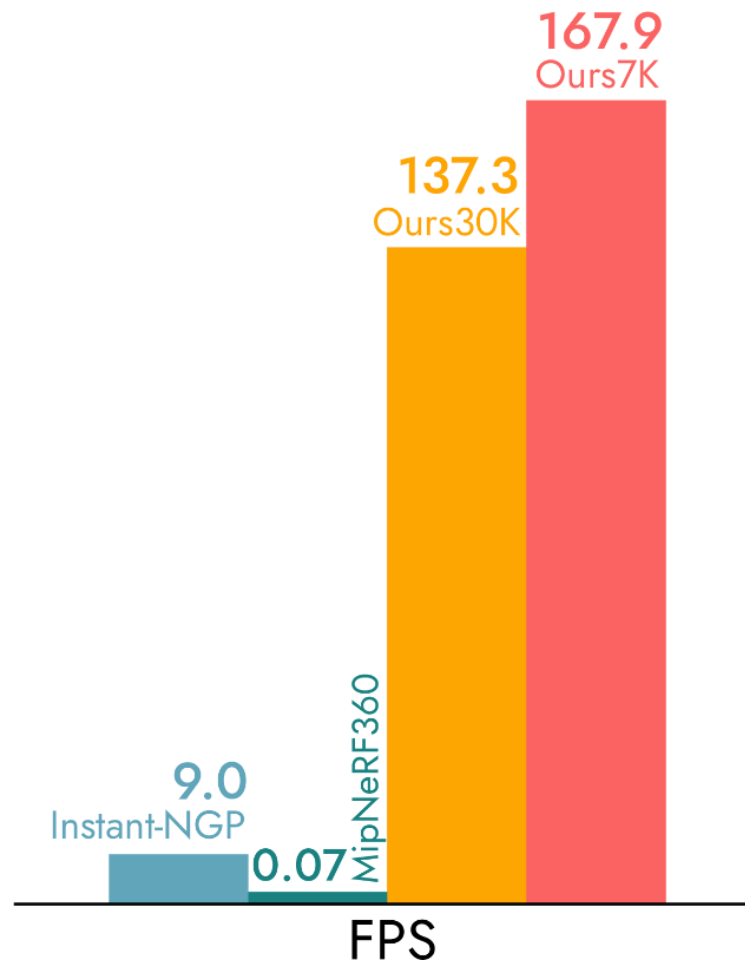
GEORGIOS KOPANAS*, Inria, Université Côte d’Azur, France

THOMAS LEIMKÜHLER, Max-Planck-Institut für Informatik, Germany

GEORGE DRETTAKIS, Inria, Université Côte d’Azur, France



Fig. 1. Our method achieves real-time rendering of radiance fields with quality that equals the previous method with the best quality [Barron et al. 2022], while only requiring optimization times competitive with the fastest previous methods [Fridovich-Keil and Yu et al. 2022; Müller et al. 2022]. Key to this performance is a novel 3D Gaussian scene representation coupled with a real-time differentiable renderer, which offers significant speedup to both scene optimization and novel view synthesis. Note that for comparable training times to InstantNGP [Müller et al. 2022], we achieve similar quality to theirs; while this is the maximum quality they reach, by training for 51min we achieve state-of-the-art quality, even slightly better than Mip-NeRF360 [Barron et al. 2022].

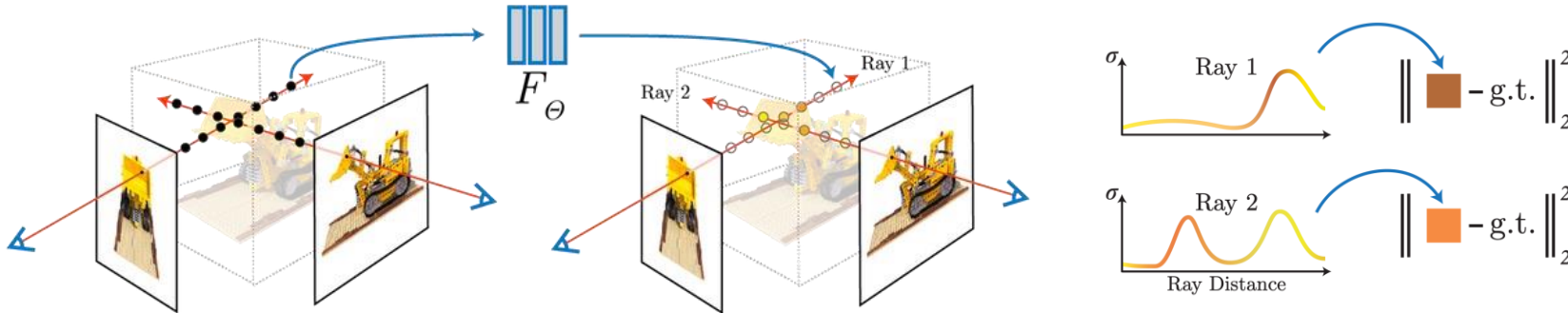


Slide adopted from 6.S980 – ML for Inverse Graphics – Vincent Sitzmann

Neural Radiance Field: Parameterize Radiance Field densely, at *every* point in space

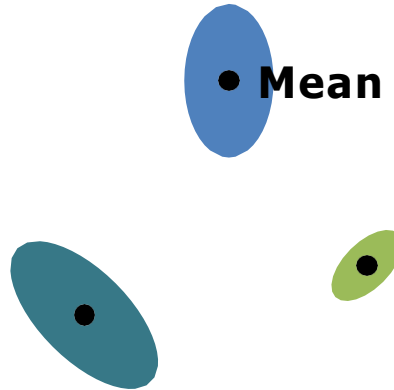
$$(x, y, z, \theta, \phi) \rightarrow \begin{array}{|c|c|c|} \hline & & \\ \hline \end{array} \rightarrow (RGB\sigma)$$

F_{Θ}



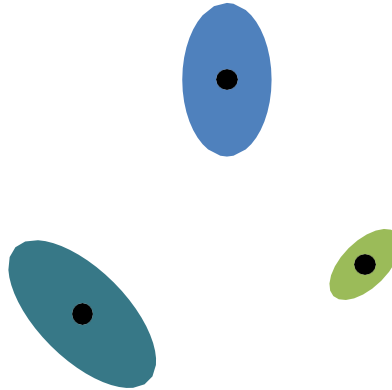
Key Idea: Parameterize Radiance Field *sparingly, only where density is nonzero*

**3D Gaussian Blobs
floating in Space**



Key Idea: Parameterize Radiance Field *sparingly, only where density is nonzero*

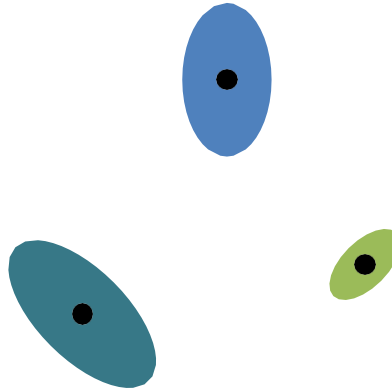
**3D Gaussian Blobs
floating in Space**



Key Idea: Parameterize Radiance Field *sparsely, only where density is nonzero*

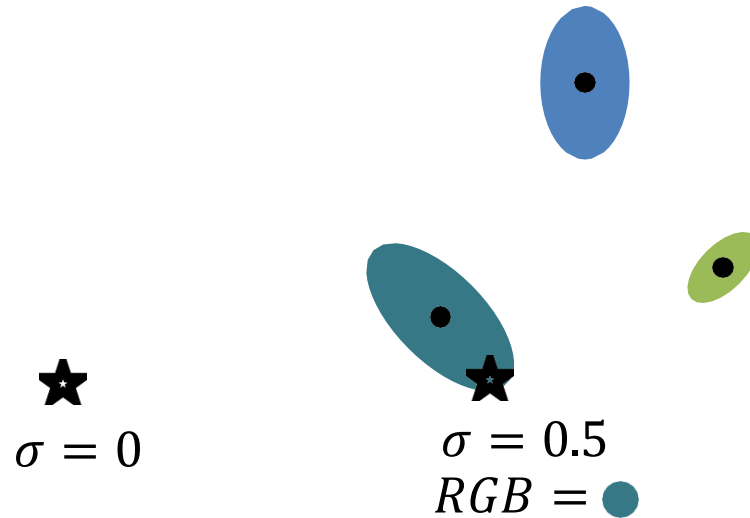
**3D Gaussian Blobs
floating in Space**

★
 $\sigma = 0$



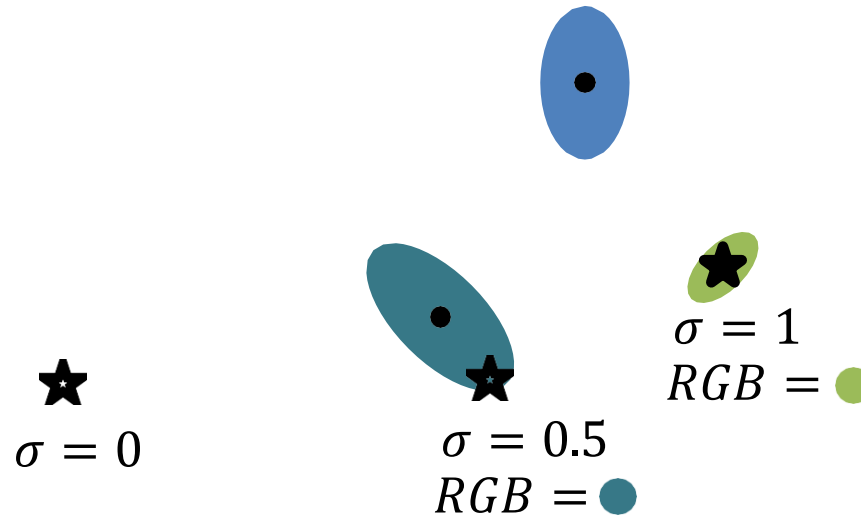
Key Idea: Parameterize Radiance Field *sparsely, only where density is nonzero*

**3D Gaussian Blobs
floating in Space**



Key Idea: Parameterize Radiance Field *sparingly, only where density is nonzero*

**3D Gaussian Blobs
floating in Space**





▼ Metrics

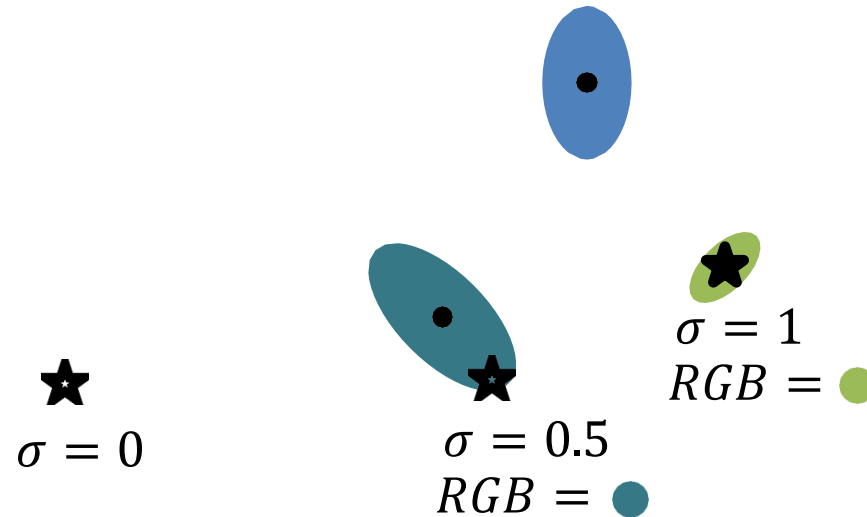
57.56 (17.37 ms)

VSync On

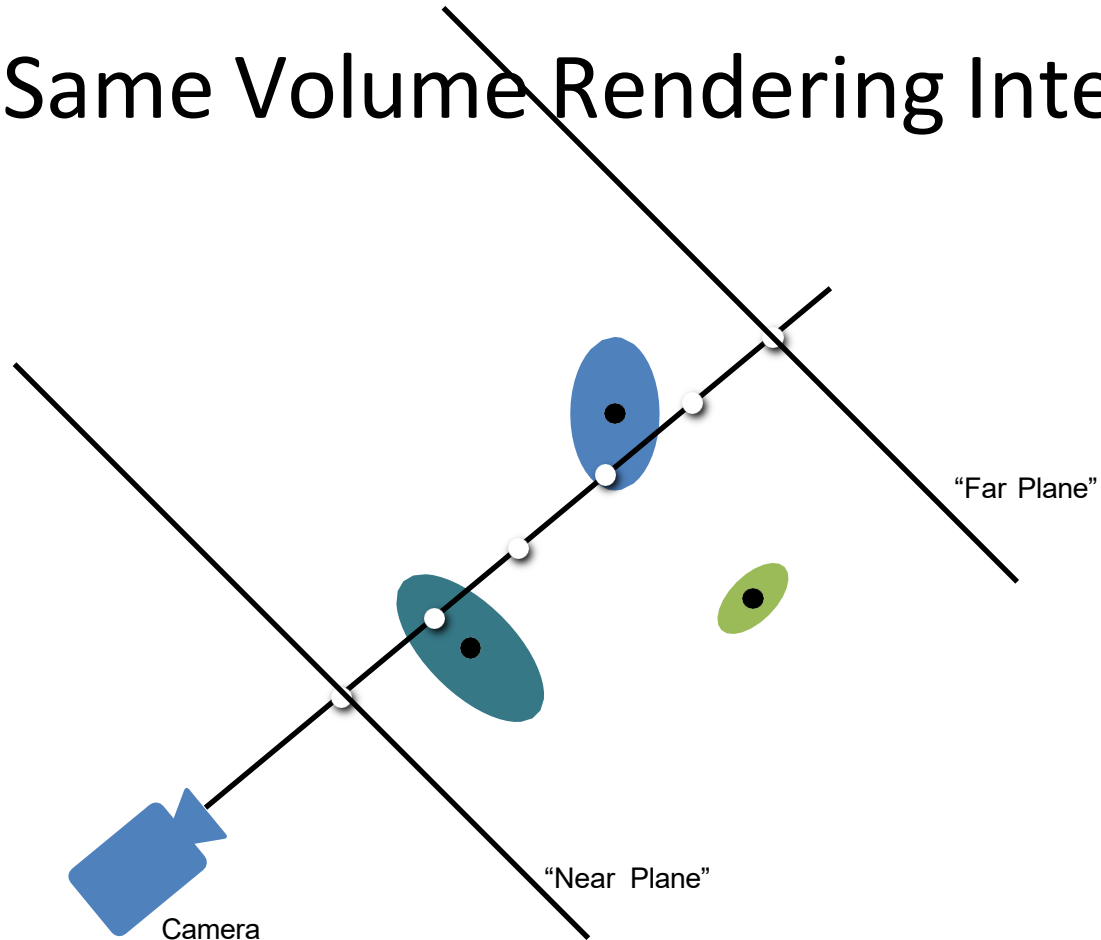


How to Render?

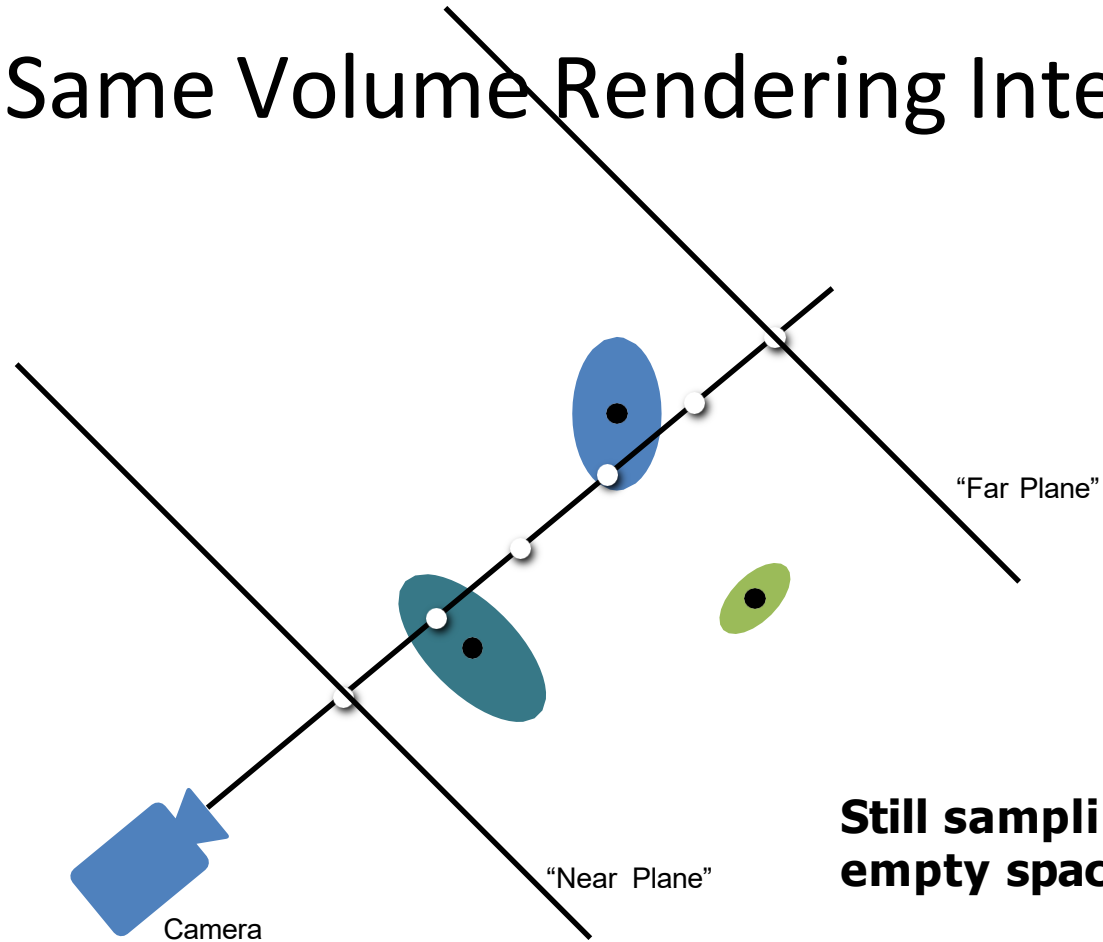
**3D Gaussian Blobs
floating in Space**



Same Volume Rendering Integral!

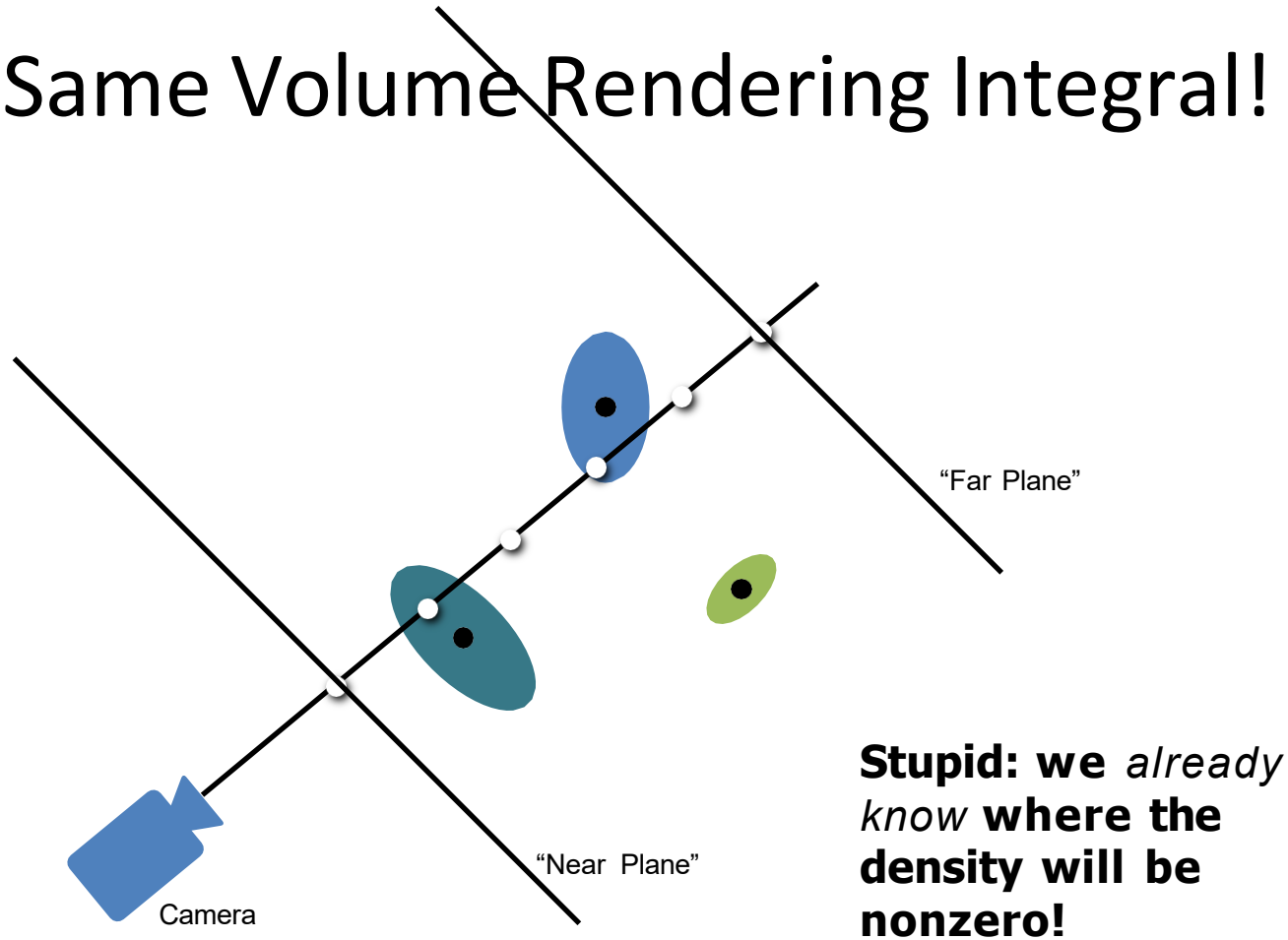


Same Volume Rendering Integral!



Still sampling lots of empty space...

Same Volume Rendering Integral!



Gaussians are closed under affine transforms, integration

$$\mathcal{G}_{\mathbf{V}}(\mathbf{x} - \mathbf{p}) = \frac{1}{2\pi|\mathbf{V}|^{\frac{1}{2}}} e^{-\frac{1}{2}(\mathbf{x} - \mathbf{p})^T \mathbf{V}^{-1}(\mathbf{x} - \mathbf{p})}$$

↑
3D Covariance!



Gaussians are closed under affine transforms, integration

$$\mathcal{G}_{\mathbf{V}}(\mathbf{x} - \mathbf{p}) = \frac{1}{2\pi|\mathbf{V}|^{\frac{1}{2}}} e^{-\frac{1}{2}(\mathbf{x} - \mathbf{p})^T \mathbf{V}^{-1}(\mathbf{x} - \mathbf{p})}$$

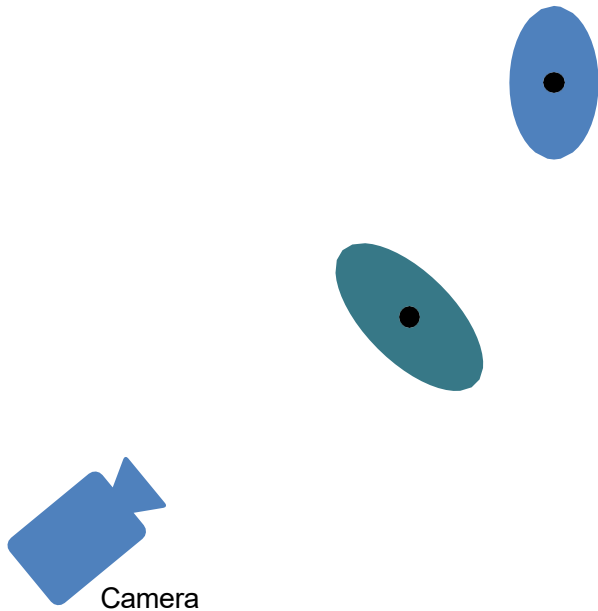
↑
3D Covariance!

Affine mapping $\Phi = \mathbf{M}\mathbf{x} + \mathbf{p}$ of coordinates
(such as cam2world matrix!):

$$\mathcal{G}_{\mathbf{V}}(\Phi^{-1}(\mathbf{u}) - \mathbf{p}) = \frac{1}{|\mathbf{M}^{-1}|} \mathcal{G}_{\mathbf{M}\mathbf{V}\mathbf{M}^T}(\mathbf{u} - \Phi(\mathbf{p}))$$



Gaussians are closed under affine transforms, integration



$$\mathcal{G}_{\mathbf{V}}(\mathbf{x} - \mathbf{p}) = \frac{1}{2\pi|\mathbf{V}|^{\frac{1}{2}}} e^{-\frac{1}{2}(\mathbf{x} - \mathbf{p})^T \mathbf{V}^{-1}(\mathbf{x} - \mathbf{p})}$$

↑ 3D Covariance!

Affine mapping $\Phi = \mathbf{M}\mathbf{x} + \mathbf{p}$ of coordinates
(such as cam2world matrix!):

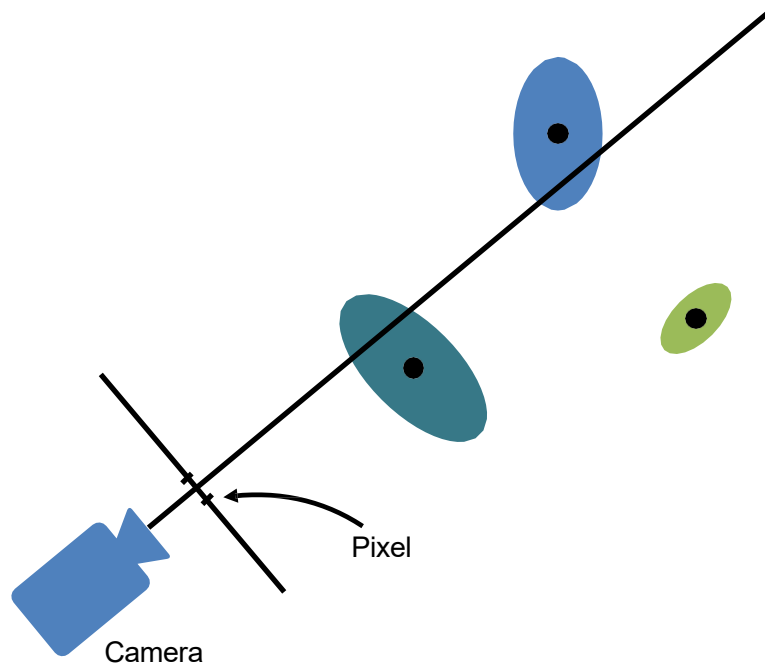
$$\mathcal{G}_{\mathbf{V}}(\Phi^{-1}(\mathbf{u}) - \mathbf{p}) = \frac{1}{|\mathbf{M}^{-1}|} \mathcal{G}_{\mathbf{M}\mathbf{V}\mathbf{M}^T}(\mathbf{u} - \Phi(\mathbf{p}))$$

Integrate along axis:

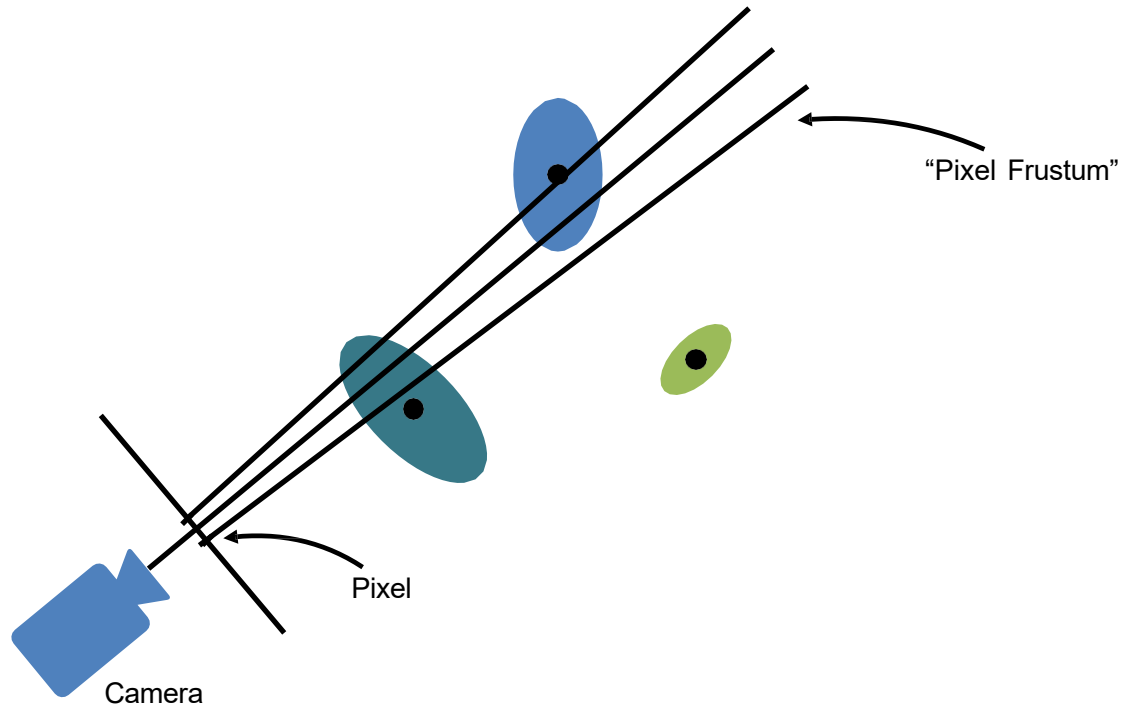
$$\int_{\mathbb{R}} \mathcal{G}_{\mathbf{V}}^3(\mathbf{x} - \mathbf{p}) dx_2 = \mathcal{G}_{\hat{\mathbf{V}}}^2(\hat{\mathbf{x}} - \hat{\mathbf{p}})$$

$$\mathbf{V} = \begin{pmatrix} a & b & c \\ b & d & e \\ c & e & f \end{pmatrix} \Leftrightarrow \begin{pmatrix} a & b \\ b & d \end{pmatrix} = \hat{\mathbf{V}}$$

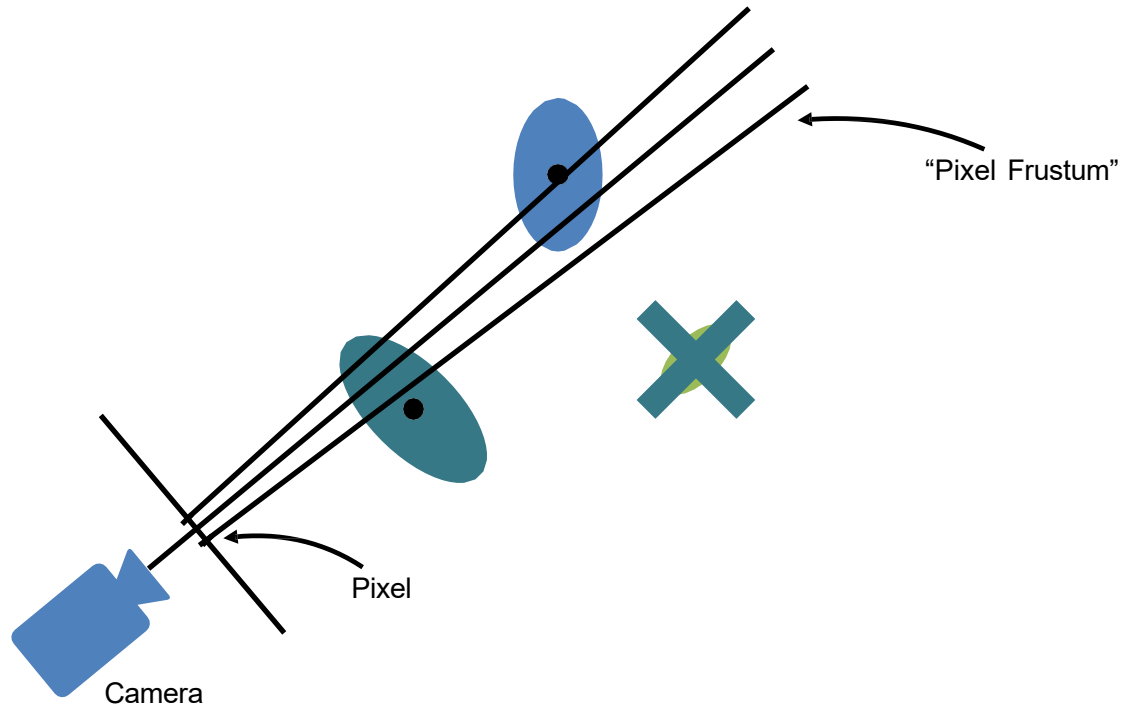
Instead: Rasterization



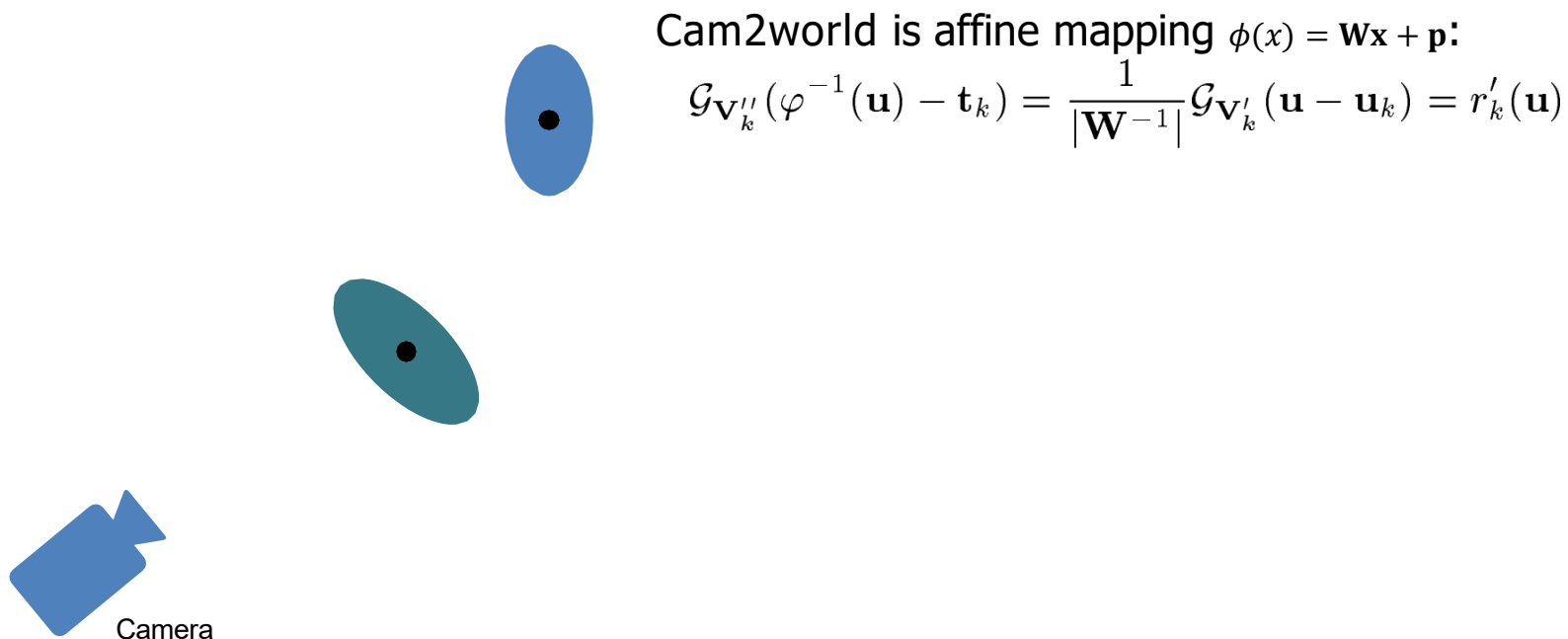
Instead: Rasterization



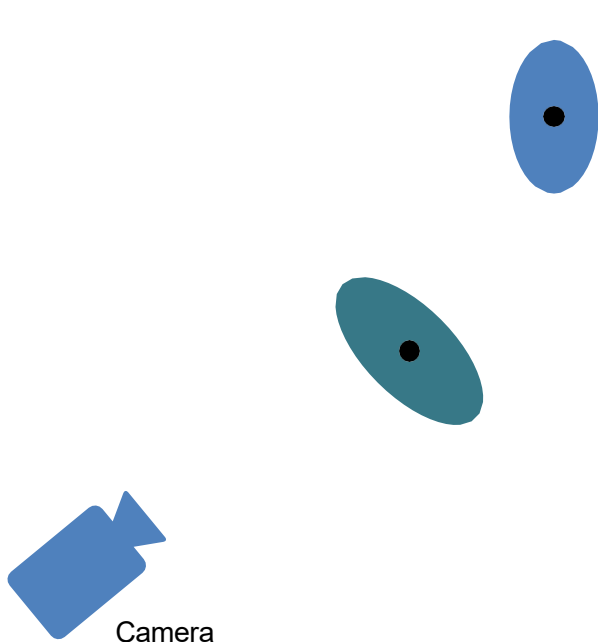
“Cull” Gaussians with less than 99% confidence relative to view frustum



Step 1: Transform Gaussians into Camera Coordinates



Step 1: Transform Gaussians into Camera Coordinates



The diagram illustrates the transformation of Gaussians into camera coordinates. It features a blue camera icon at the bottom left, labeled "Camera". Two ellipses are shown: a light blue one at the top and a teal one below it, both containing a black dot representing the center of the Gaussian. The ellipses are tilted, representing their orientation in 3D space.

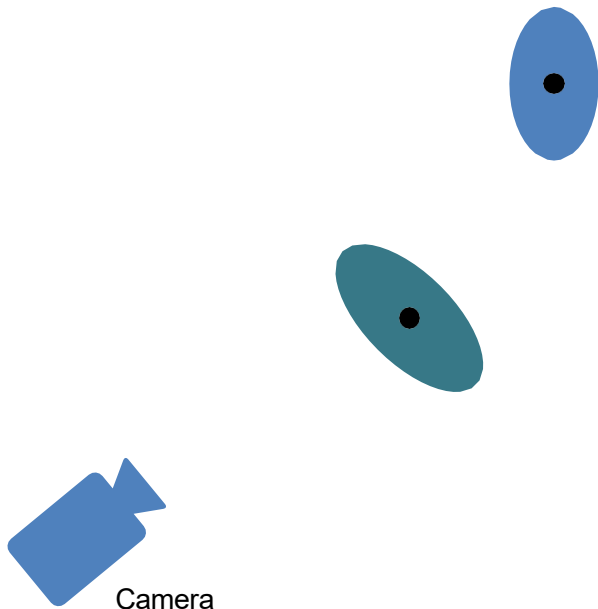
Cam2world is affine mapping $\phi(x) = \mathbf{W}\mathbf{x} + \mathbf{p}$:

$$\mathcal{G}_{\mathbf{V}_k''}(\varphi^{-1}(\mathbf{u}) - \mathbf{t}_k) = \frac{1}{|\mathbf{W}^{-1}|} \mathcal{G}_{\mathbf{V}_k'}(\mathbf{u} - \mathbf{u}_k) = r'_k(\mathbf{u})$$

Projection $\mathbf{m}(u)$ is *not* an affine mapping :/

$$\begin{pmatrix} x_0 \\ x_1 \\ x_2 \end{pmatrix} = \mathbf{m}(\mathbf{u}) = \begin{pmatrix} u_0/u_2 \\ u_1/u_2 \\ \|(u_0, u_1, u_2)^T\| \end{pmatrix}$$
$$\begin{pmatrix} u_0 \\ u_1 \\ u_2 \end{pmatrix} = \mathbf{m}^{-1}(\mathbf{x}) = \begin{pmatrix} x_0/l \cdot x_2 \\ x_1/l \cdot x_2 \\ 1/l \cdot x_2 \end{pmatrix},$$

Step 1: Transform Gaussians into Camera Coordinates



Cam2world is affine mapping $\phi(x) = \mathbf{W}\mathbf{x} + \mathbf{p}$:

$$\mathcal{G}_{\mathbf{V}_k''}(\varphi^{-1}(\mathbf{u}) - \mathbf{t}_k) = \frac{1}{|\mathbf{W}^{-1}|} \mathcal{G}_{\mathbf{V}_k'}(\mathbf{u} - \mathbf{u}_k) = r'_k(\mathbf{u})$$

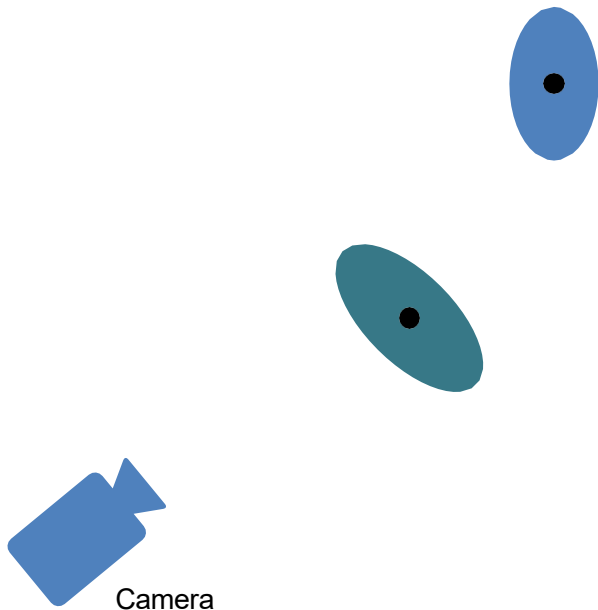
Projection $\mathbf{m}(u)$ is *not* an affine mapping :/

$$\begin{pmatrix} x_0 \\ x_1 \\ x_2 \end{pmatrix} = \mathbf{m}(\mathbf{u}) = \begin{pmatrix} u_0/u_2 \\ u_1/u_2 \\ \|(u_0, u_1, u_2)^T\| \end{pmatrix}$$
$$\begin{pmatrix} u_0 \\ u_1 \\ u_2 \end{pmatrix} = \mathbf{m}^{-1}(\mathbf{x}) = \begin{pmatrix} x_0/l \cdot x_2 \\ x_1/l \cdot x_2 \\ 1/l \cdot x_2 \end{pmatrix},$$

But can approximate with first-order Taylor Expansion as:

$$\mathbf{m}_{\mathbf{u}_k}(\mathbf{u}) = \mathbf{x}_k + \mathbf{J}_{\mathbf{u}_k} \cdot (\mathbf{u} - \mathbf{u}_k) \quad \mathbf{J}_{\mathbf{u}_k} = \frac{\partial \mathbf{m}}{\partial \mathbf{u}}(\mathbf{u}_k)$$

Step 1: Transform Gaussians into Camera Coordinates



Cam2world is affine mapping $\phi(x) = \mathbf{W}\mathbf{x} + \mathbf{p}$:

$$\mathcal{G}_{\mathbf{V}_k''}(\varphi^{-1}(\mathbf{u}) - \mathbf{t}_k) = \frac{1}{|\mathbf{W}^{-1}|} \mathcal{G}_{\mathbf{V}_k'}(\mathbf{u} - \mathbf{u}_k) = r'_k(\mathbf{u})$$

Projection $\mathbf{m}(u)$ is *not* an affine mapping :/

$$\begin{pmatrix} x_0 \\ x_1 \\ x_2 \end{pmatrix} = \mathbf{m}(\mathbf{u}) = \begin{pmatrix} u_0/u_2 \\ u_1/u_2 \\ \|(u_0, u_1, u_2)^T\| \end{pmatrix}$$

$$\begin{pmatrix} u_0 \\ u_1 \\ u_2 \end{pmatrix} = \mathbf{m}^{-1}(\mathbf{x}) = \begin{pmatrix} x_0/l \cdot x_2 \\ x_1/l \cdot x_2 \\ 1/l \cdot x_2 \end{pmatrix},$$

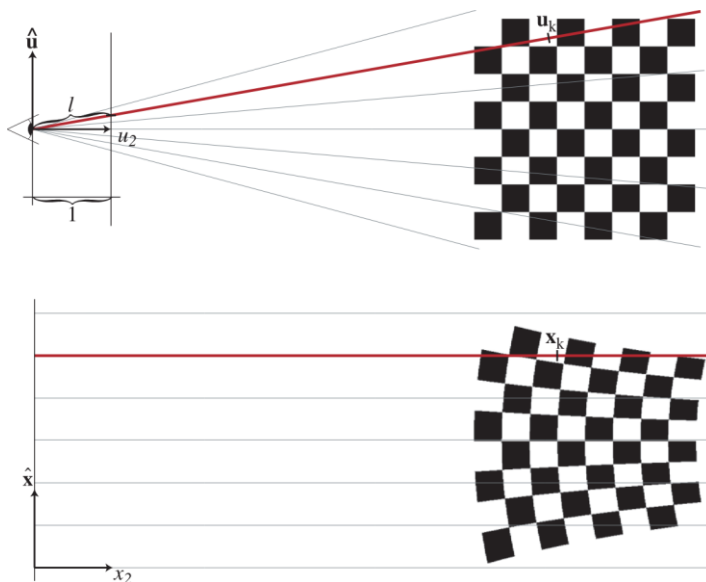
But can approximate with first-order Taylor Expansion as:

$$\mathbf{m}_{\mathbf{u}_k}(\mathbf{u}) = \mathbf{x}_k + \mathbf{J}_{\mathbf{u}_k} \cdot (\mathbf{u} - \mathbf{u}_k) \quad \mathbf{J}_{\mathbf{u}_k} = \frac{\partial \mathbf{m}}{\partial \mathbf{u}}(\mathbf{u}_k)$$

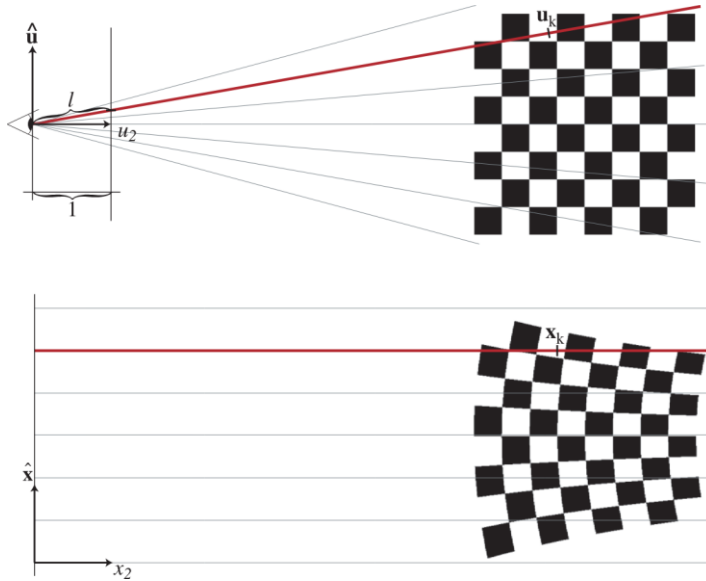
Step 1: Transform Gaussians into Camera Coordinates

But can approximate with first-order Taylor Expansion as:

$$\mathbf{m}_{\mathbf{u}_k}(\mathbf{u}) = \mathbf{x}_k + \mathbf{J}_{\mathbf{u}_k} \cdot (\mathbf{u} - \mathbf{u}_k) \quad \mathbf{J}_{\mathbf{u}_k} = \frac{\partial \mathbf{m}}{\partial \mathbf{u}}(\mathbf{u}_k)$$



Step 1: Transform Gaussians into Camera Coordinates



But can approximate with first-order Taylor Expansion as:

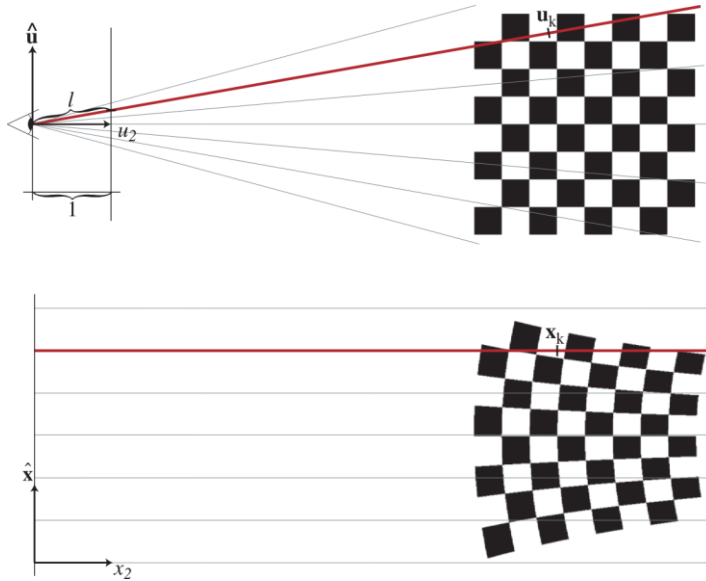
$$\mathbf{m}_{\mathbf{u}_k}(\mathbf{u}) = \mathbf{x}_k + \mathbf{J}_{\mathbf{u}_k} \cdot (\mathbf{u} - \mathbf{u}_k) \quad \mathbf{J}_{\mathbf{u}_k} = \frac{\partial \mathbf{m}}{\partial \mathbf{u}}(\mathbf{u}_k)$$

Projected, 2D Gaussians are then:

$$\frac{1}{|\mathbf{W}^{-1}| |\mathbf{J}^{-1}|} \mathcal{G}_{\mathbf{V}_k}(\mathbf{x} - \mathbf{x}_k)$$

$$\begin{aligned} \mathbf{V}_k &= \mathbf{J} \mathbf{V}'_k \mathbf{J}^T \\ &= \mathbf{J} \mathbf{W} \mathbf{V}''_k \mathbf{W}^T \mathbf{J}^T. \end{aligned}$$

Step 1: Transform Gaussians into Camera Coordinates



But can approximate with first-order Taylor Expansion as:

$$\mathbf{m}_{\mathbf{u}_k}(\mathbf{u}) = \mathbf{x}_k + \mathbf{J}_{\mathbf{u}_k} \cdot (\mathbf{u} - \mathbf{u}_k) \quad \mathbf{J}_{\mathbf{u}_k} = \frac{\partial \mathbf{m}}{\partial \mathbf{u}}(\mathbf{u}_k)$$

Projected, 2D Gaussians are then:

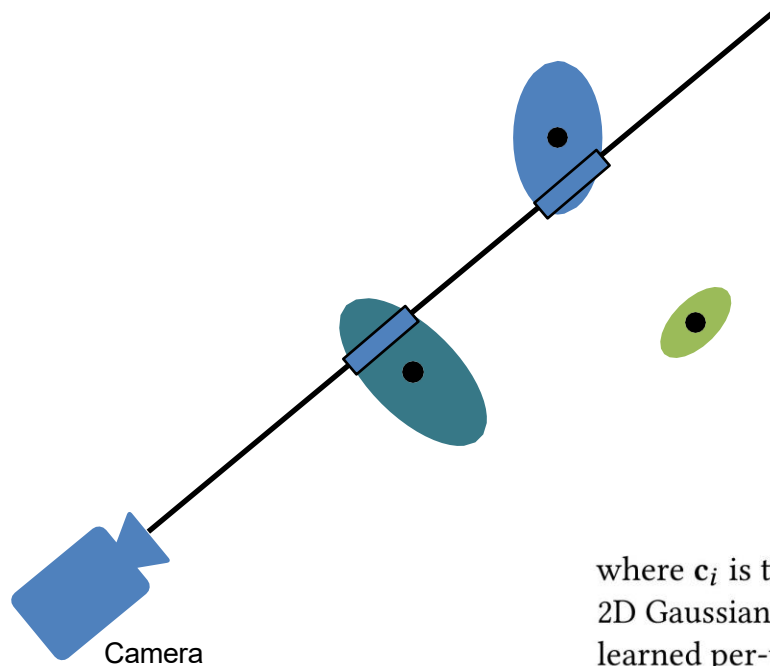
$$\frac{1}{|\mathbf{W}^{-1}| |\mathbf{J}^{-1}|} \mathcal{G}_{\mathbf{V}_k}(\mathbf{x} - \mathbf{x}_k)$$

$$\begin{aligned} \mathbf{V}_k &= \mathbf{J} \mathbf{V}'_k \mathbf{J}^T \\ &= \mathbf{J} \mathbf{W} \mathbf{V}''_k \mathbf{W}^T \mathbf{J}^T. \end{aligned}$$

Finally, can integrate along rays:

$$\begin{aligned} q_k(\hat{\mathbf{x}}) &= \int_{\mathbb{R}} \frac{1}{|\mathbf{J}^{-1}| |\mathbf{W}^{-1}|} \mathcal{G}_{\mathbf{V}_k}(\hat{\mathbf{x}} - \hat{\mathbf{x}}_k, x_2 - x_{k2}) dx_2 \\ &= \frac{1}{|\mathbf{J}^{-1}| |\mathbf{W}^{-1}|} \mathcal{G}_{\hat{\mathbf{V}}_k}(\hat{\mathbf{x}} - \hat{\mathbf{x}}_k), \end{aligned} \quad (21)$$

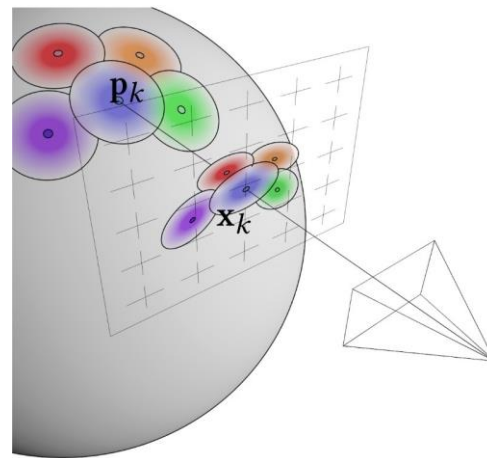
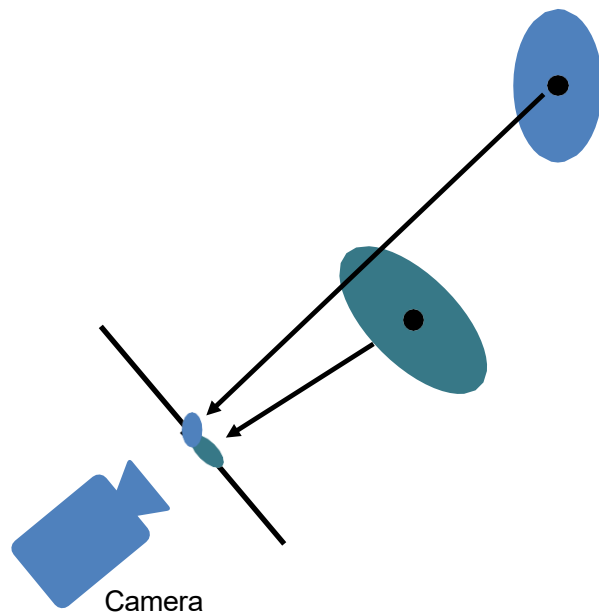
Can compute volume rendering integral without ever sampling a single 3D point in space!



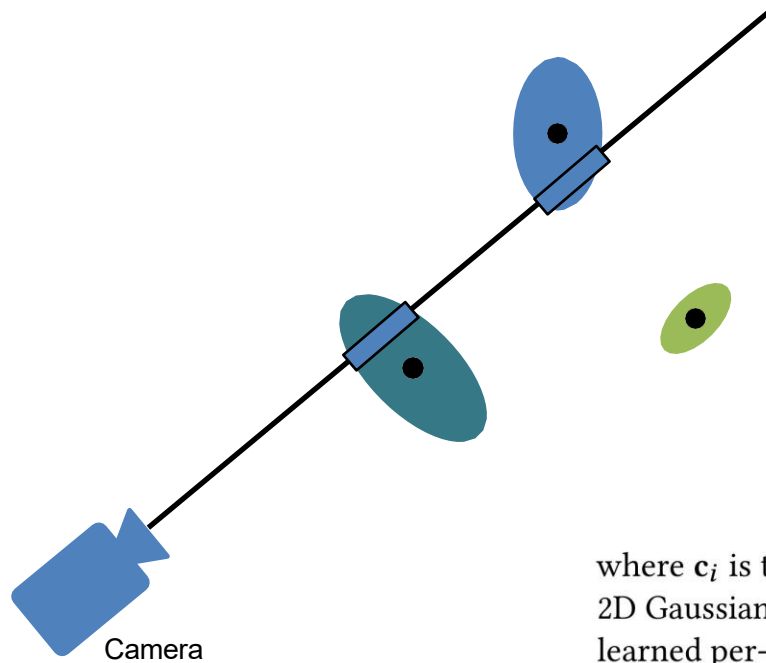
$$C = \sum_{i \in \mathcal{N}} c_i \alpha_i \prod_{j=1}^{i-1} (1 - \alpha_j), \quad (3)$$

where c_i is the color of each point and α_i is given by evaluating a 2D Gaussian with covariance Σ [Yifan et al. 2019] multiplied with a learned per-point opacity.

Projected 3D Gaussian makes 2D Gaussian!



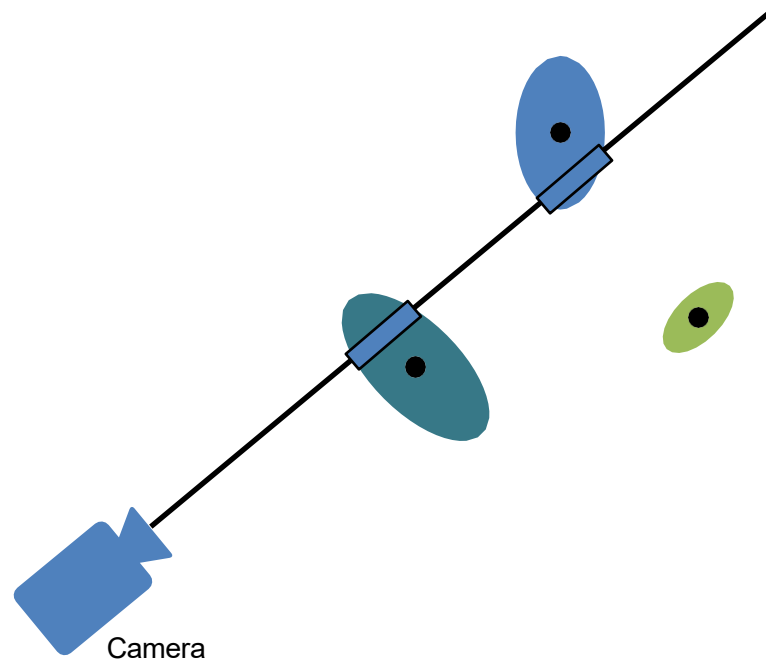
Any problems for inverse graphics, though...?



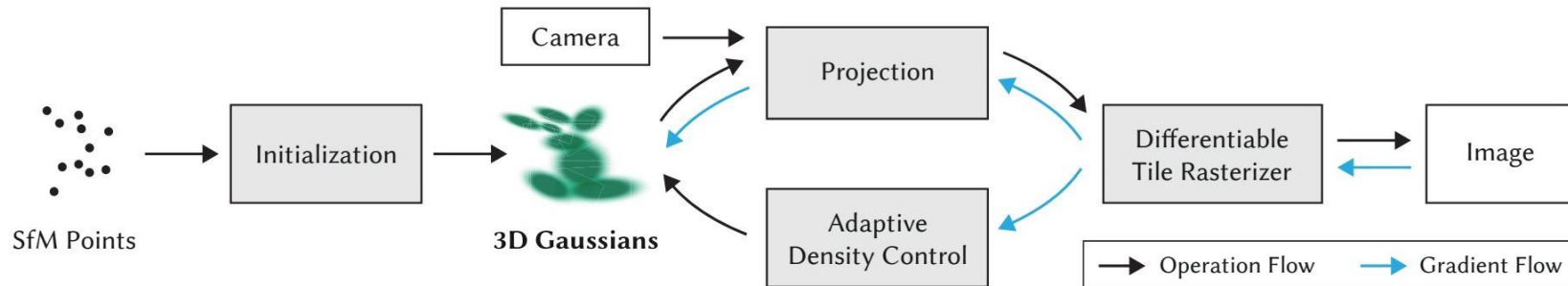
$$C = \sum_{i \in \mathcal{N}} c_i \alpha_i \prod_{j=1}^{i-1} (1 - \alpha_j), \quad (3)$$

where c_i is the color of each point and α_i is given by evaluating a 2D Gaussian with covariance Σ [Yifan et al. 2019] multiplied with a learned per-point opacity.

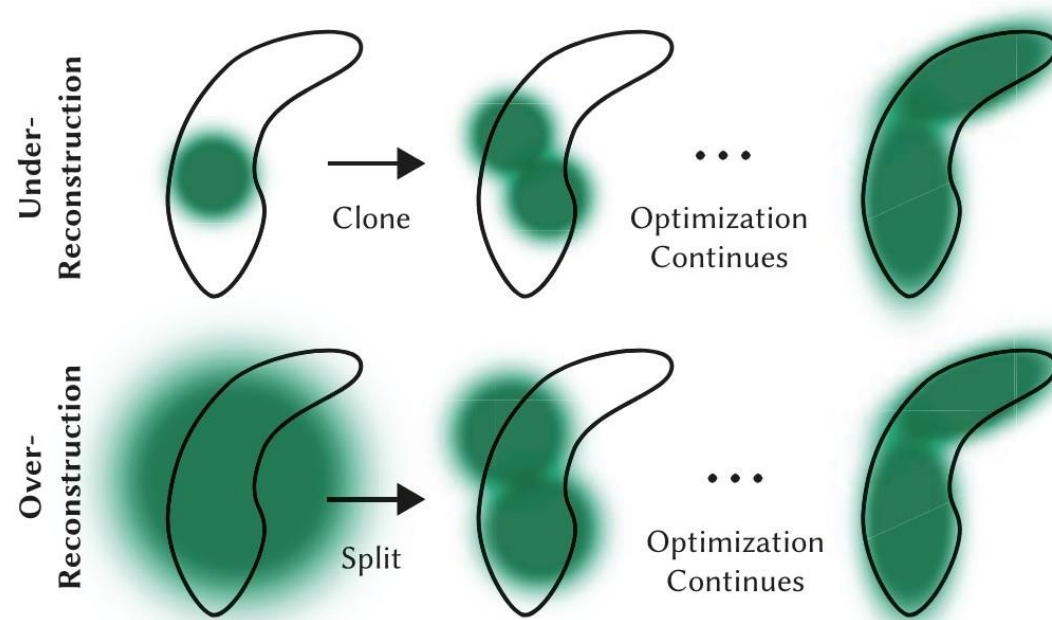
Problem: Local minima...



Fix 1: Start from SfM point cloud.



Fix 2: Heuristic *pruning* and *spawning* operations





This video contains a voice-over

3D Gaussian Splatting for Real-Time Radiance Field Rendering

SIGGRAPH 2023
(ACM Transactions on Graphics)

Bernhard Kerbl*



Georgios Kopanas*



Thomas Leimkühler



George Drettakis



* Denotes equal contribution

The End